Airport Management System

Passenger Service Management Use Cases Documentation

Prepared by:

Betül Kaan

Course:

Software Modeling and Design

Date:

26 March 2026

Document Overview

This document presents a structured set of System Administration models for the Airport Management System. It includes:

✔ Viewing *Flight Schedules (UC-01)*

✔ *Uploading Flight Documents (UC-02)*

✔ *Online Check-In (UC-03)*

✔ *Boarding Pass Download (UC-04)*

✔ *Checking Gate Information (UC-05)*

✔ *Tracking Baggage Status (UC-06)*

✔ *Receiving Flight Notification (UC-07)*

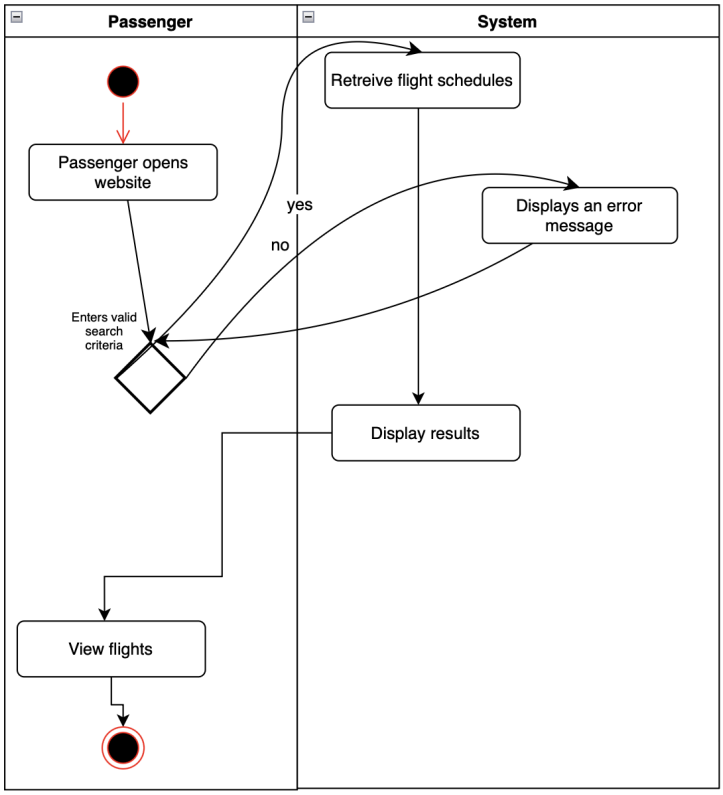
Each use case is defined with:

1. ***Mapped Functional Requirements: Linking diagrams to specific UC IDs.***
2. ***Mapped Alternative Flows: Modeling system behavior during errors or threats.***
3. ***Non-functional Requirements: Defining performance and security constraints.***
4. ***Swimlane Actor Interactions: Visualizing the flow between Passengers, Staff, and the System.***

The goal of this document is to clearly model and describe administrative system behavior in a professional and structured way.

Activity Diagrams:

UC-01: View Flight Schedules



Core Purpose:

This use case describes how passengers search for and view available flight schedules based on their travel criteria.

Functional Requirements Mapped:

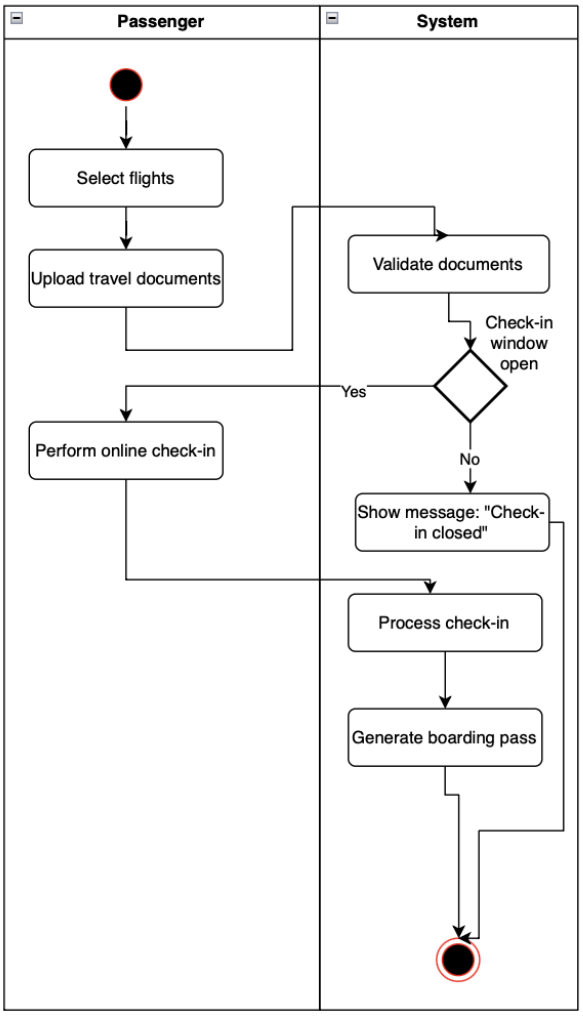
- *Passenger enters search criteria (destination, date, etc.).*
- *System retrieves available flight schedules from the database.*
- *System displays matching flights to the passenger.*
- *Passenger browses and reviews flight options.*

Alternative Flow Mapped:

If no flights match the search criteria, the system displays a "no results found" message and prompts the passenger to modify their search.

UC-02: Upload Travel Documents

UC-03: Perform Online Check-In



Core Purpose:

This use case describes how passengers complete the online check-in process and obtain a boarding pass.

Functional Requirements Mapped:

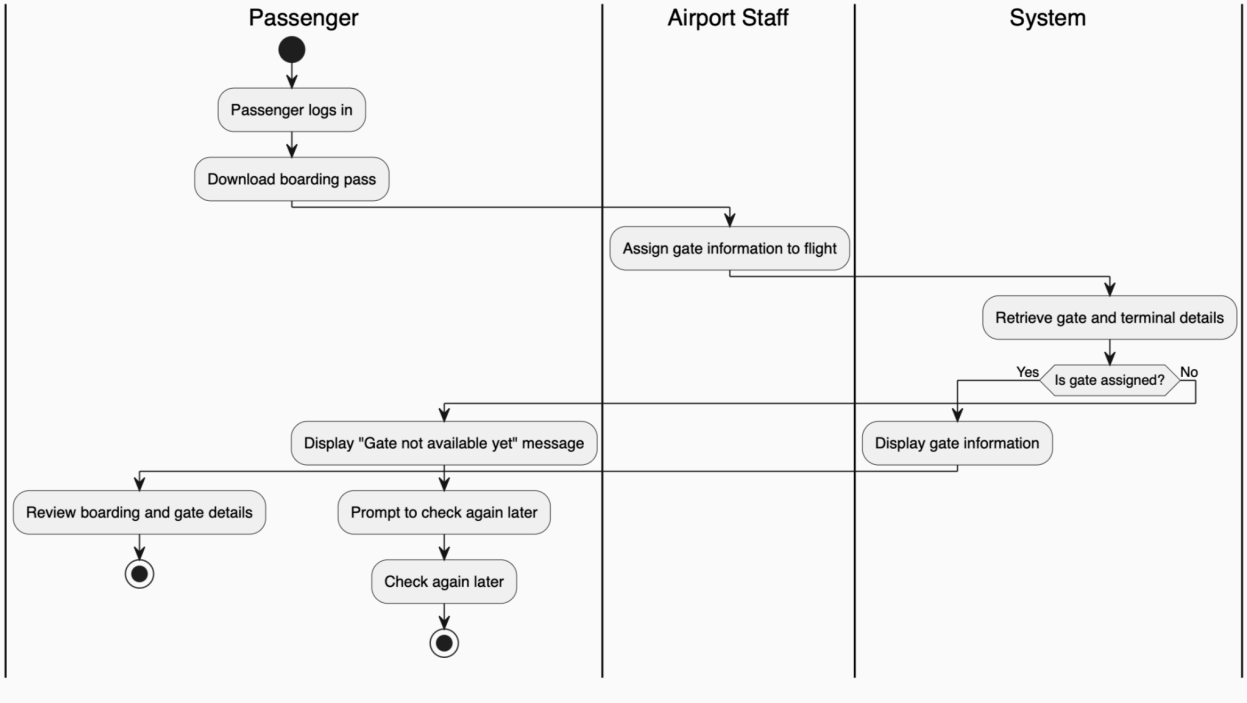
- *Passenger selects a booked flight.*
- *Passenger uploads required travel documents.*
- *System validates the submitted documents.*
- *Passenger initiates online check-in.*
- *System processes check-in and verifies eligibility.*
- *System checks whether the check-in window is open.*
- *System generates a boarding pass upon successful check-in.*

Alternative Flow Mapped:

- *If documents are invalid, the system prompts the passenger to re-upload correct documents.*
- *If the check-in window is closed, the system notifies the passenger that online check-in is unavailable.*

UC-04 Download Boarding Pass

UC-05 Check Assigned Gate Information



Core Purpose:

This use case describes how passengers access their boarding pass and retrieve gate and terminal information.

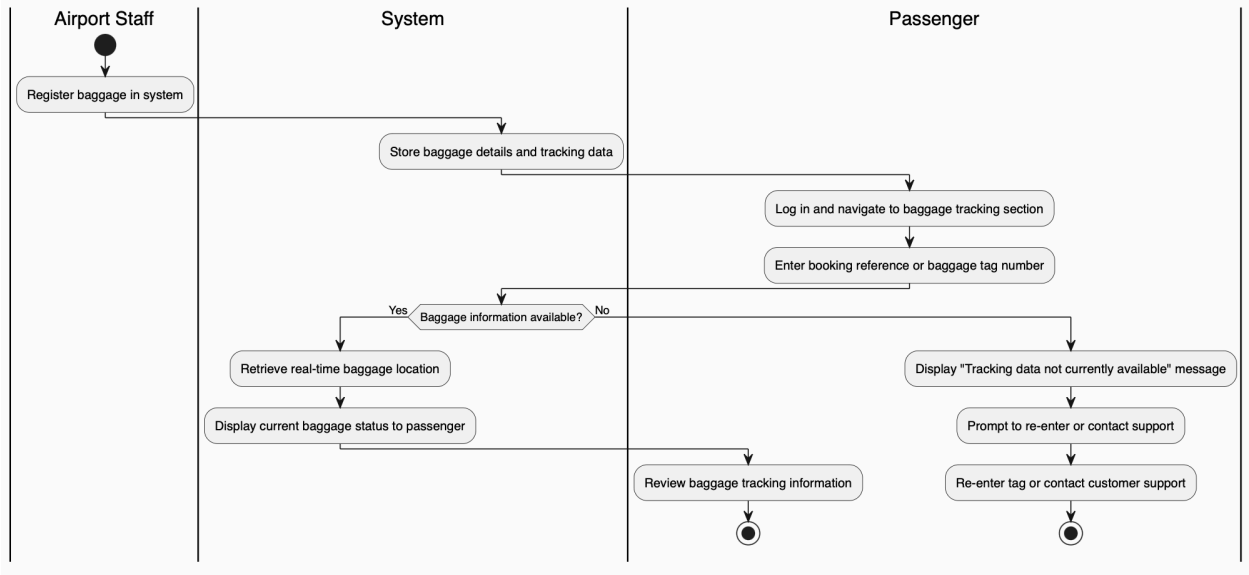
Functional Requirements Mapped:

- *Passenger logs into the system.*
- *Passenger downloads the boarding pass.*
- *Airport staff assigns gate information to the flight.*
- *System retrieves gate and terminal details.*
- *System displays updated gate information to the passenger.*
- *Passenger reviews boarding and gate details.*

Alternative Flow Mapped:

If gate information is not yet assigned, the system displays a "Gate not available yet" message and prompts the passenger to check again later.

UC-06 Track Baggage Status



Core Purpose:

This use case describes how passengers track the status and location of their checked baggage.

Functional Requirements Mapped:

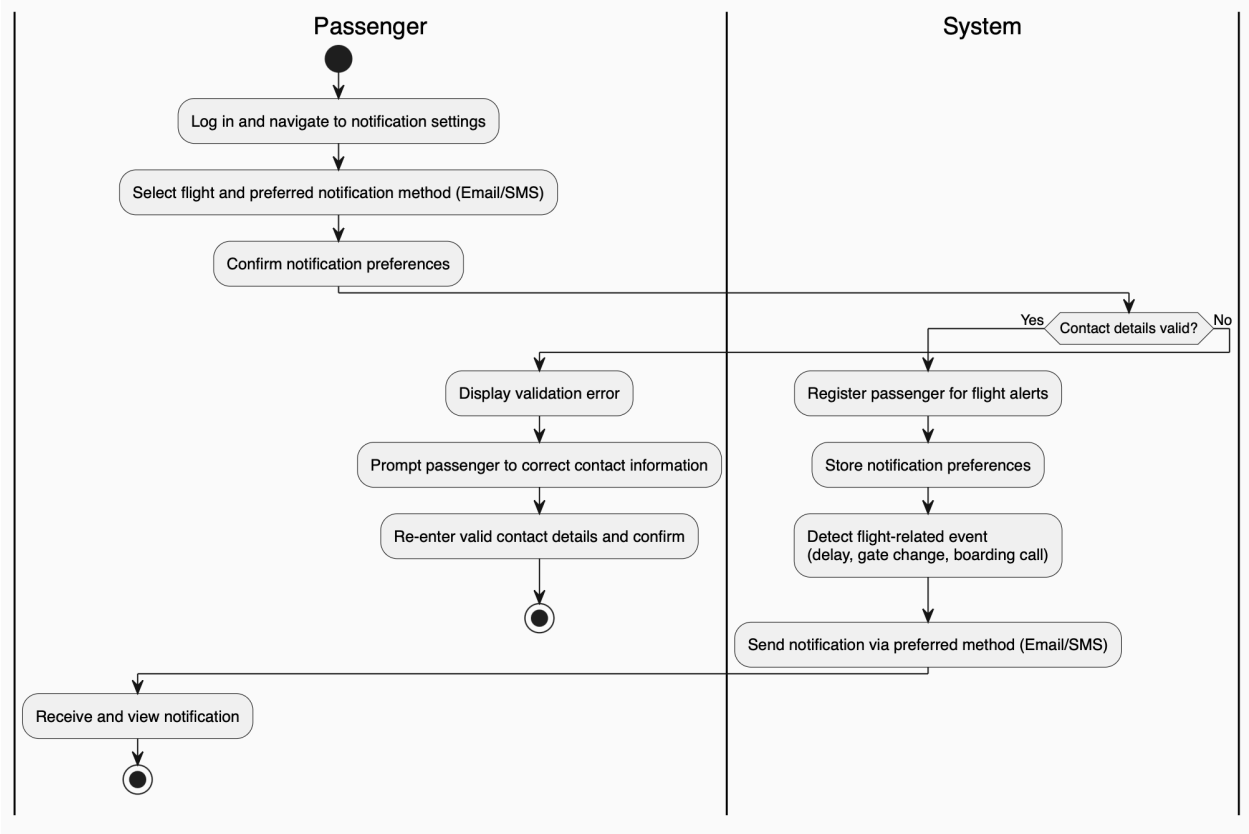
- *Airport staff registers baggage in the system.*

- *System stores baggage details and tracking data.*
- *Passenger requests baggage status.*
- *System retrieves real-time baggage location.*
- *System displays current baggage status to the passenger.*

Alternative Flow Mapped:

If baggage information is unavailable or not yet registered, the system notifies the passenger that tracking data is not currently available.

UC-07 Receive Flight Notifications



Core Purpose:

This use case describes how passengers receive real-time notifications regarding their flight.

Functional Requirements Mapped:

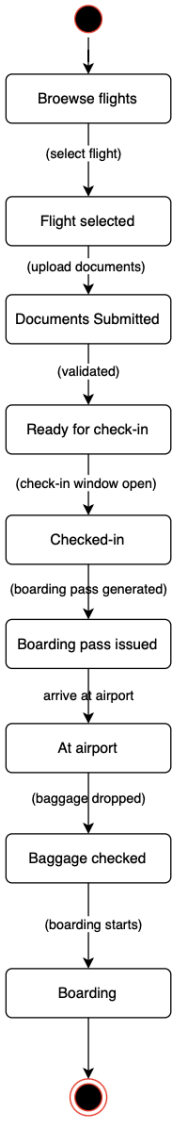
- *Passenger sets notification preferences (e.g., SMS, email).*
- *System stores passenger preferences.*
- *System detects flight-related events (delays, gate changes, etc.).*
- *System sends notifications based on stored preferences.*

- *Passenger receives and views notifications.*

Alternative Flow Mapped:

If notification delivery fails (e.g., invalid contact details), the system prompts the passenger to update their contact information.

State Diagram:



Core Purpose:

This state diagram describes the lifecycle of a passenger's flight journey, from browsing flights to completing the trip, showing how the passenger transitions between different states in the system.

Functional Requirements Mapped:

- *Passenger can browse and select flights.*
- *System transitions passenger from Browsing Flights to Flight Selected upon selection.*
- *Passenger submits required travel documents.*
- *System validates documents and updates state accordingly.*
- *Passenger becomes eligible for check-in once documents are approved.*
- *System verifies check-in window availability.*
- *Passenger completes check-in and receives a boarding pass.*
- *Passenger proceeds to the airport and may check baggage.*
- *Passenger transitions to boarding when boarding begins.*
- *System marks journey as completed after flight departure.*

State Transitions Mapped:

- *Browsing Flights → Flight Selected (flight chosen)*
- *Flight Selected → Documents Submitted (documents uploaded)*
- *Documents Submitted → Ready for Check-in (documents valid)*
- *Ready for Check-in → Checked-in (check-in successful)*
- *Checked-in → Boarding Pass Issued (boarding pass generated)*
- *Boarding Pass Issued → At Airport (passenger arrives)*
- *At Airport → Baggage Checked (baggage registered)*
- *Baggage Checked → Boarding (boarding starts)*
- *Boarding → Flight Completed (journey ends)*

Alternative Flow Mapped:

- *If documents are invalid, the passenger transitions to Documents Rejected and must resubmit valid documents.*
- *If the check-in window is closed, the passenger transitions to Check-in Failed and cannot proceed with online check-in.*
- *If baggage is not checked, the passenger may transition directly from At Airport to Boarding.*

Airport Management System

Prepared by:

Albina Blloshmi

Course:

Software Modeling and Design

Date:

27 March 2026

Baggage Management System – Behavioral Models

✔ Baggage Check-In & Screening Activity Diagram (UC-08)

✔ Baggage Tracking Activity Diagram (UC-09)

✔ Baggage Lifecycle State Diagram (UC-08 → UC-13)

✔ Each model is defined with:

1. *Mapped Functional Requirements*
2. *Mapped Alternative Flows*
3. *Non-functional Requirements*
4. *Actor Interactions*

The goal of this document is to clearly model and describe the system's behavioral workflows in a professional and structured way.

Core Purpose:

To manage baggage throughout its lifecycle from check-in to final delivery, ensuring accurate tracking, handling of delays and lost baggage, and efficient delivery to passengers.

This use case describes how passengers claim their baggage upon arrival at the destination airport.

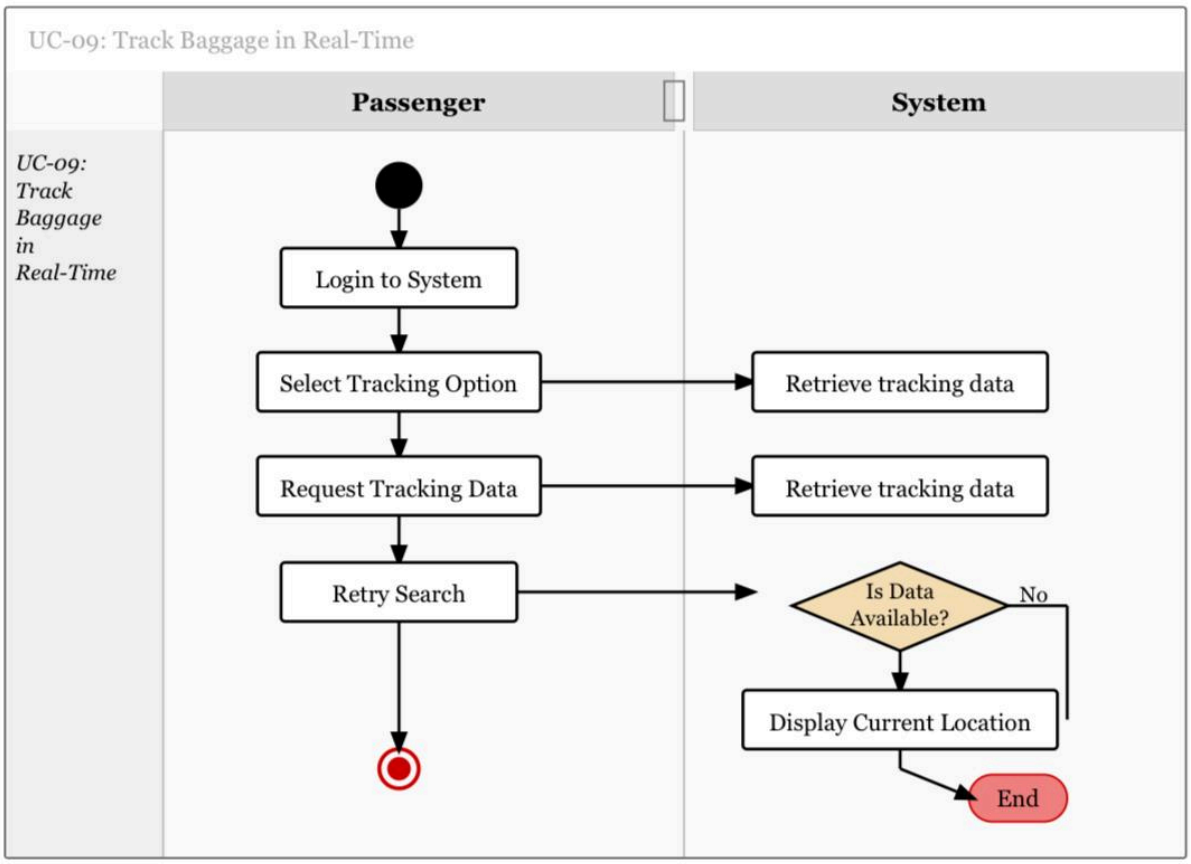
Functional Requirements Mapped:

- *Passenger goes to the baggage claim area to pick up their baggage.*
- *System displays carousel info to the passenger.*
- *Passenger collects their baggage or reports missing baggage if not found.*

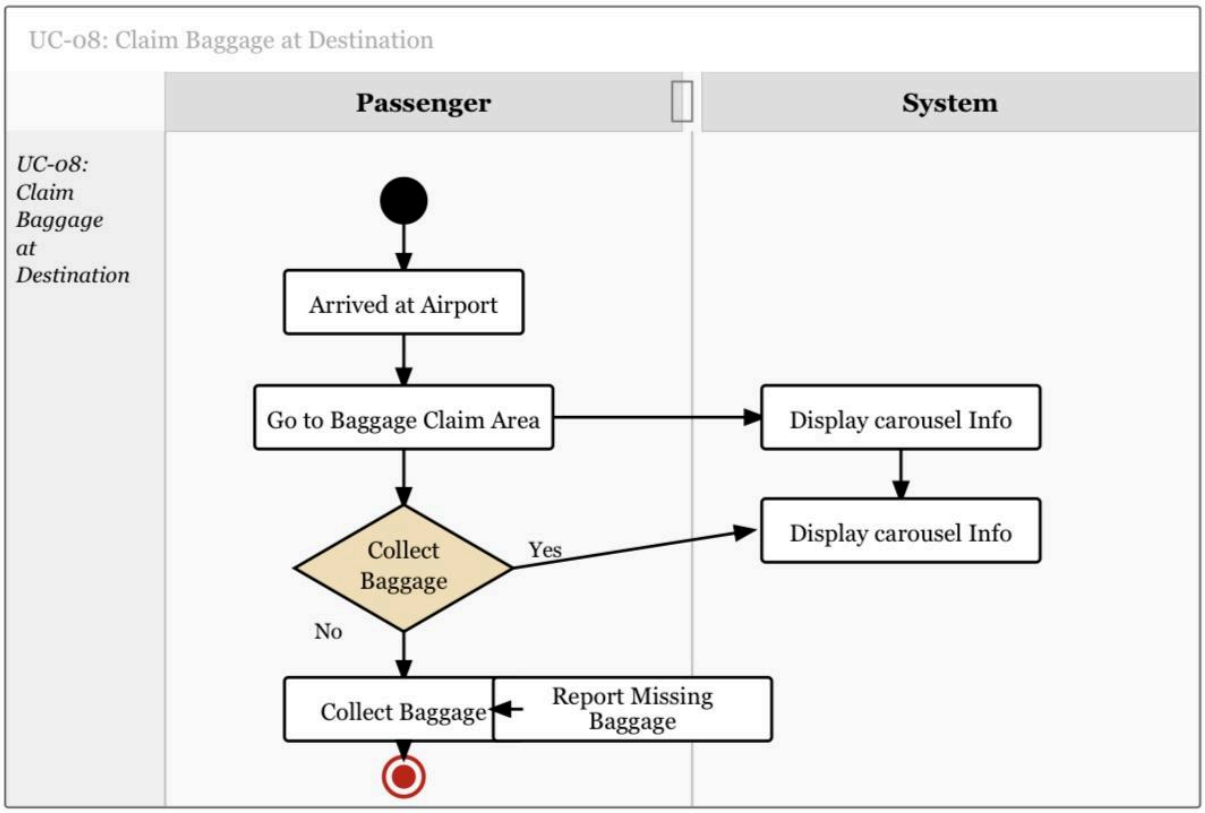
Alternative Flow Mapped:

If the passenger's baggage is not found, the system allows the passenger to report missing baggage to airport staff.

Activity Diagram:



Activity Diagram:



Core Purpose:

Core Purpose:

This use case describes how passengers track the real-time location of their checked baggage during the journey.

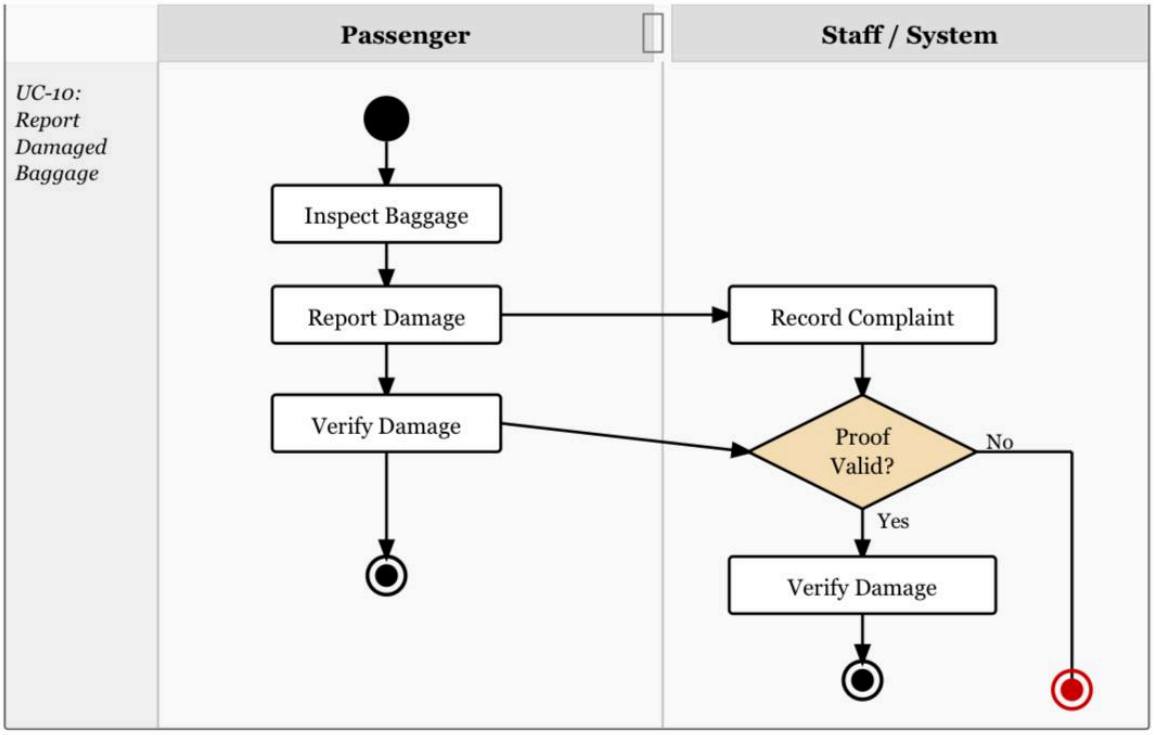
Functional Requirements Mapped:

- *Passenger logs into the tracking system.*
- *Passenger selects the tracking option.*
- *System retrieves real-time tracking data and displays the current location of the baggage.*

Alternative Flow Mapped:

If tracking data is not available, the system shows the last known location and prompts the passenger to retry the search.

Activity Diagram:



Core Purpose:

This use case describes how passengers report damage to their baggage upon arrival.

Functional Requirements Mapped:

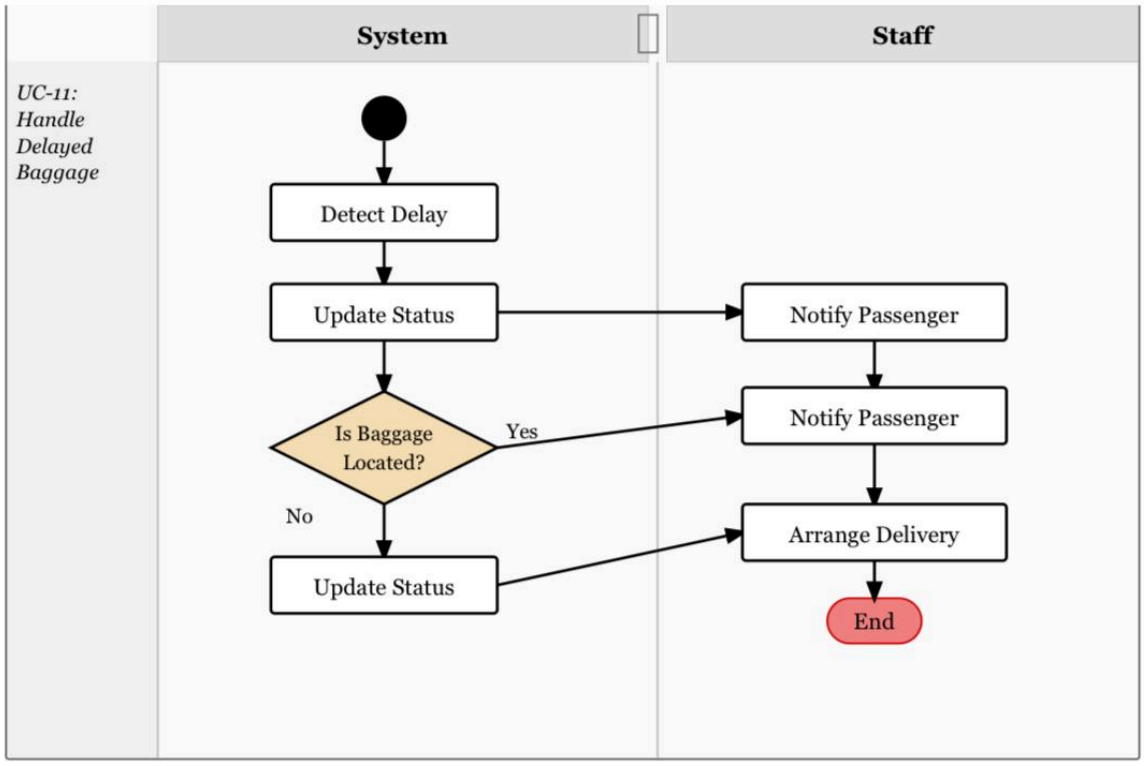
- *Passenger inspects their baggage for damage.*
- *Passenger reports baggage damage via system or counter desk.*
- *System records the complaint, verifies proof, and processes the damage claim.*

Alternative Flow Mapped:

If the proof of damage is invalid, the system rejects the claim and notifies the passenger.

Activity Diagram:

UC-11: Handle Delayed Baggage



Core Purpose:

This use case describes how the system handles cases when a passenger's baggage is delayed.

Functional Requirements Mapped:

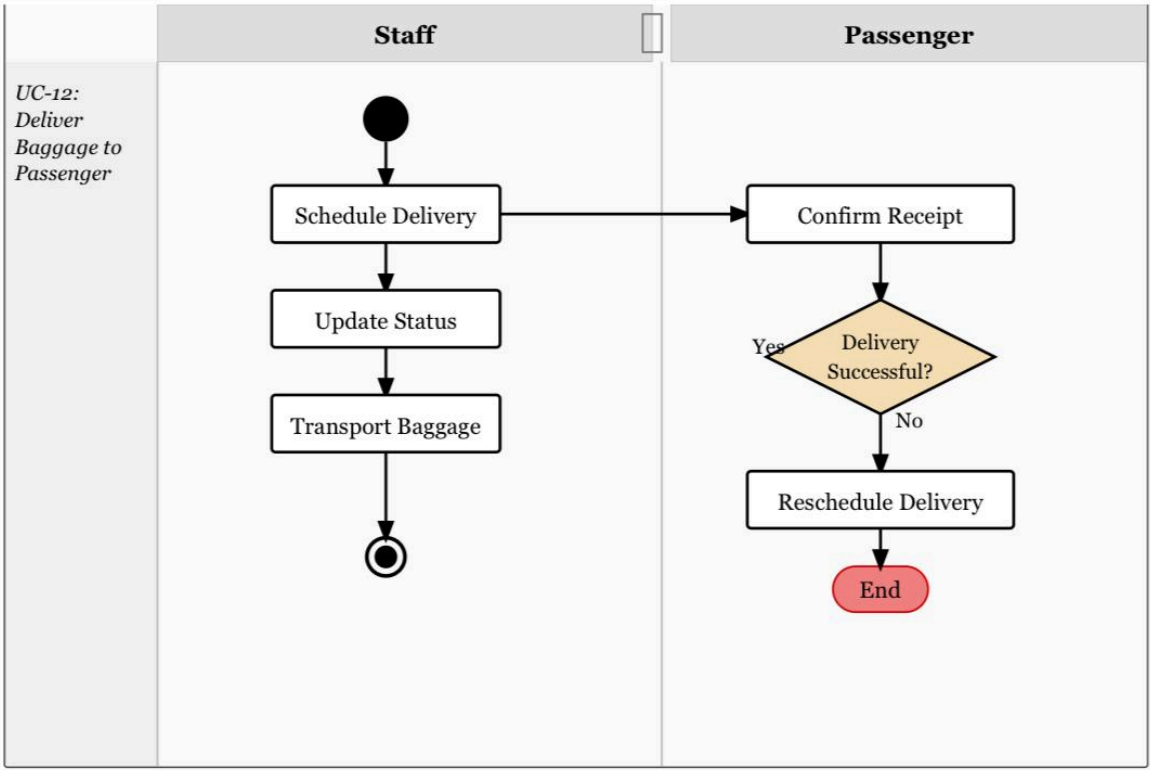
- *System detects when baggage is delayed.*
- *System updates the baggage status to indicate delay.*
- *Passenger is notified and staff arrange for delivery.*

Alternative Flow Mapped:

If baggage cannot be located, the system marks it as lost and initiates the lost baggage procedure.

Activity Diagram:

UC-12: Deliver Baggage to Passenger



Core Purpose:

This use case describes how staff delivers baggage to passengers after a delay or rerouting.

Functional Requirements Mapped:

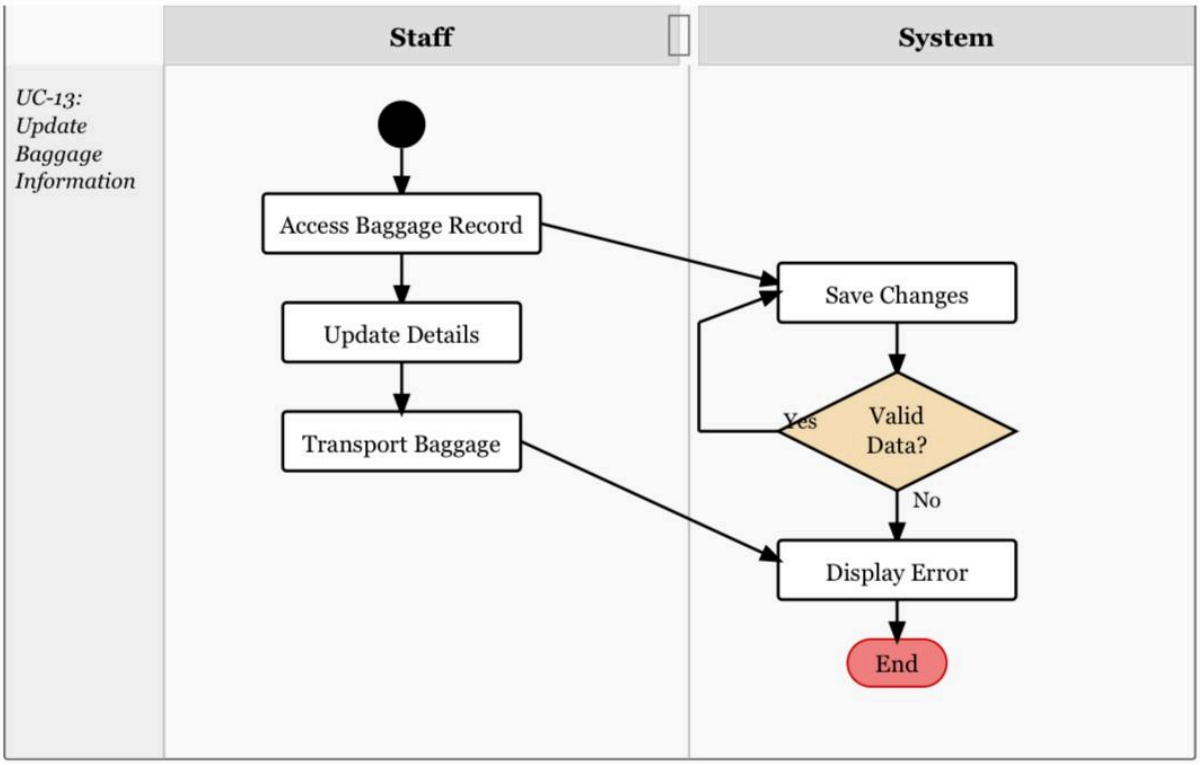
- *Staff schedules the delivery of located baggage.*
- *System updates the delivery status.*
- *Baggage is transported to the passenger's location.*
- *Passenger confirms receipt of the baggage.*

Alternative Flow Mapped:

If the delivery fails, staff reschedules the delivery and attempts it again.

Activity Diagram:

UC-13: Update Baggage Information



Core Purpose:

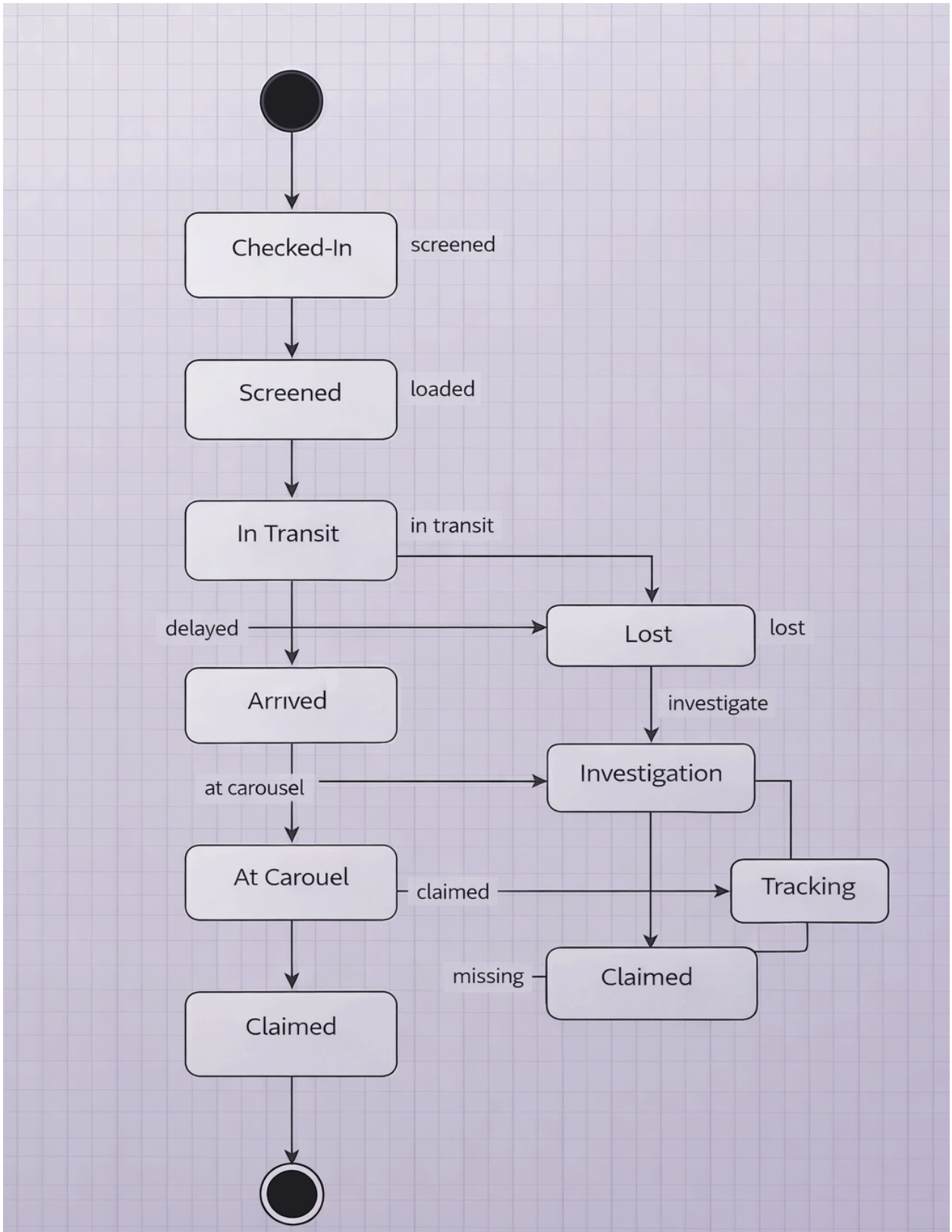
This use case describes how staff updates baggage details within the system.

Functional Requirements Mapped:

- *Staff accesses existing baggage record through the system.*
- *Staff updates the baggage details.*
- *System saves the changes and reflects the updated information.*

Alternative Flow Mapped:

If invalid data is entered, the system displays an error message and halts the update process.



Baggage State Diagram for UC-08 to UC-13

Functional Requirements Mapped:

- ***UC-08 (Baggage Check-In & Screening):***

  Register baggage, attach identification, and route it for security screening.

- ***UC-09 (Baggage Tracking):***

  Monitor baggage status in real time throughout the journey.

- ***UC-10 (Lost Baggage Handling):***

  Report, investigate, and recover lost baggage.

- ***UC-11 (Delayed Baggage Handling):***

  Detect delays, notify passengers, and manage delayed baggage.

- ***UC-12 (Baggage Delivery Service):***

  Arrange delivery of baggage when not claimed at the airport.

- ***UC-13 (Baggage Status Update):***

  Continuously update baggage information at each stage.

Alternative Flow Mapped:

- ***Delayed Baggage (UC-11):***

  If baggage is delayed, it is marked as Delayed and tracked until resolved.

- ***Lost Baggage (UC-10):***

  If baggage cannot be located, it moves to Investigation and may later be recovered.

- ***Missing at Carousel (UC-12):***

  If baggage is not claimed, it is marked as Missing and tracked.

-

- ***Status Updates (UC-13):***

  All status changes are automatically updated in the system.

Airport Management System

Flight Operations Management Use Cases Documentation

Prepared by:

Alvi Elezi

Course:

Software Modeling and Design

Date:

20 March 2026

Document Overview

This document presents a set of Flight Operations Management Activity Diagrams for the Airport Management System.

It includes:

✓ Flight status updates

✓ Flight delay management

✓ Departure time updates

✓ Arrival time updates

✓ Coordination between boarding processes

✓ Flight operations monitoring

✓ Each use case is defined with:

✓ Actors

✓ Preconditions

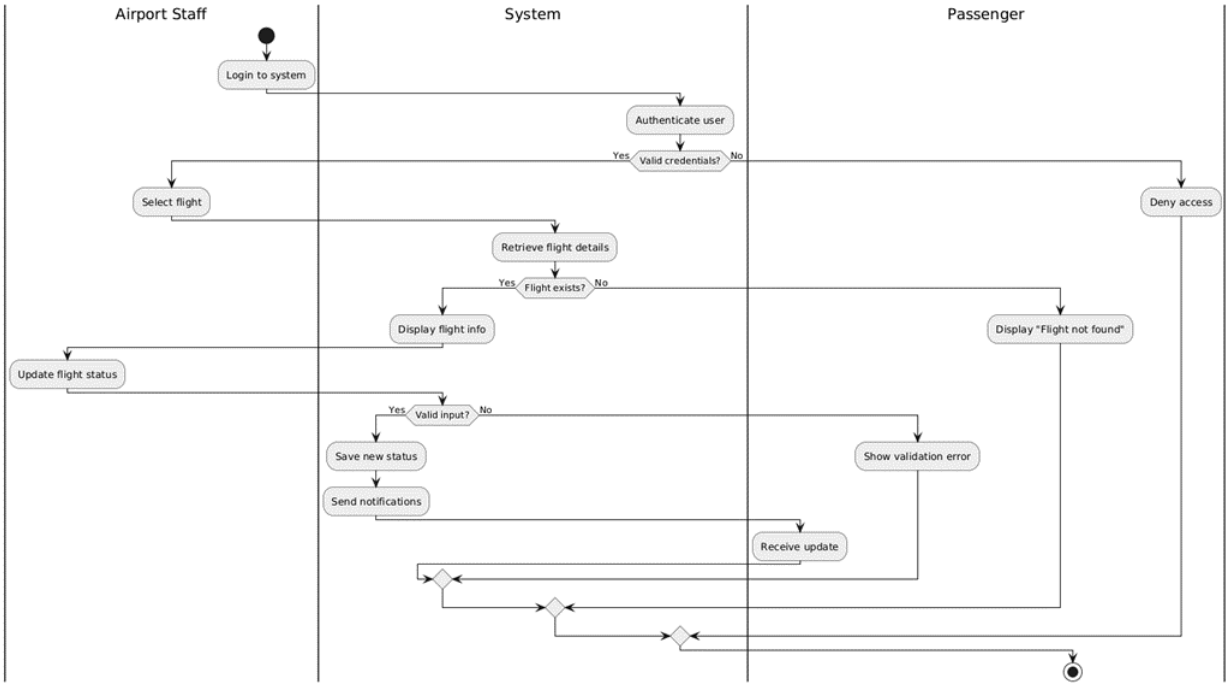
✓ Main and alternative flows

✓ Non-functional requirements

✓ Postconditions

The goal of this document is to model and describe the management of flight operations by airport staff and the behavior of the system in a structured way.

Update flight status



Core Purpose: *This activity diagram describes how airport staff update the operational status of a flight and notify relevant stakeholders.*

Functional Requirements Mapped:

Airport staff updates flight status.

System retrieves and displays flight data.

System validates updates and propagates changes across modules.

System sends notifications to passengers.

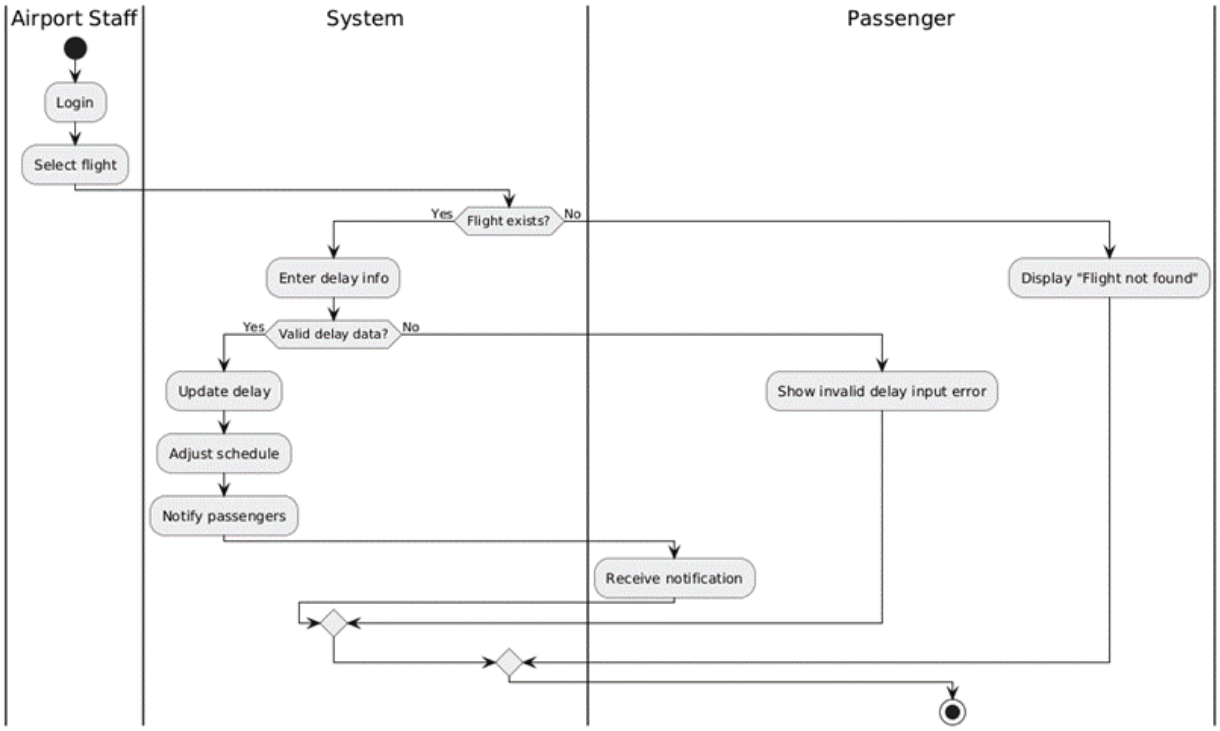
Alternative Flow Mapped:

If login credentials are invalid, access is denied.

If the selected flight does not exist, the system displays an error.

If invalid status data is entered, the system prompts the staff to correct the input.

Manage flight delays



Core Purpose: *This activity diagram describes how airport staff manage and record flight delays while ensuring passengers are informed.*

Functional Requirements Mapped:

Airport staff manages flight delays.

System updates flight schedule based on delay.

System records delay reason.

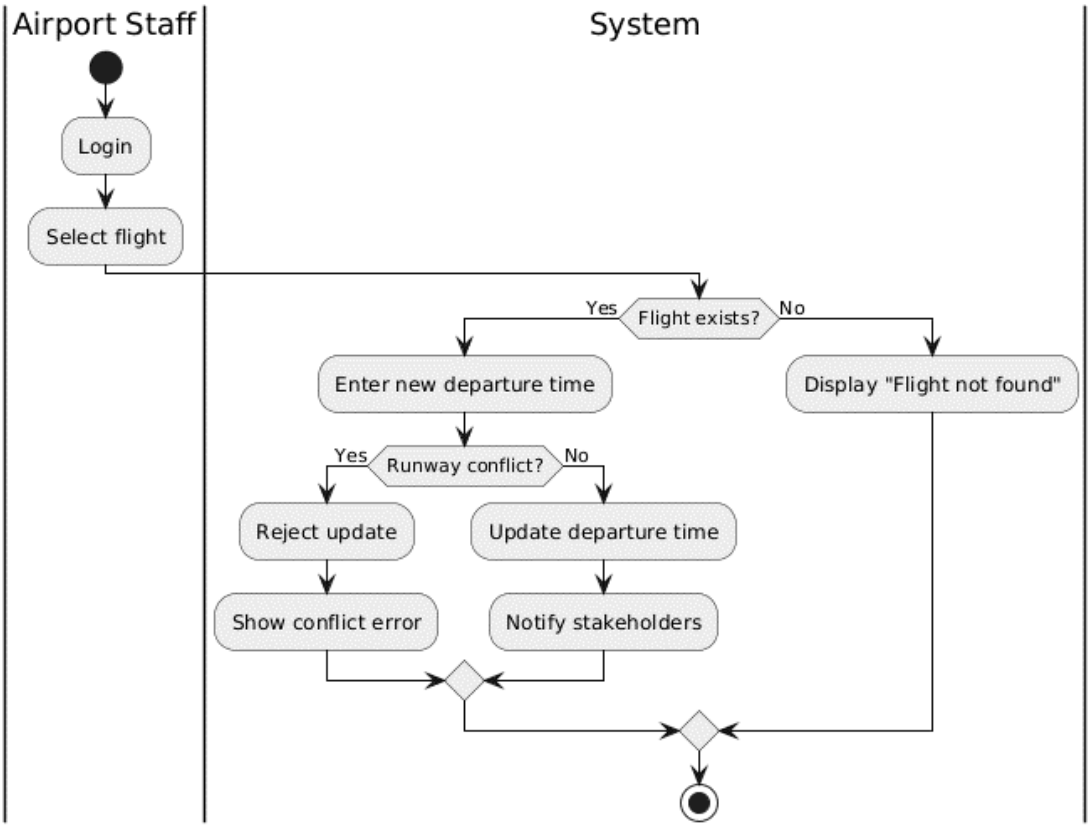
System notifies passengers of delays.

Alternative Flow Mapped:

If the flight does not exist, the system returns an error.

If invalid delay data is entered, the system prompts correction.

Update departure time



Core Purpose*: This activity diagram describes how airport staff update the scheduled departure time of a flight.*

Functional Requirements Mapped:

Airport staff updates departure times.

System validates scheduling constraints.

System updates flight schedule.

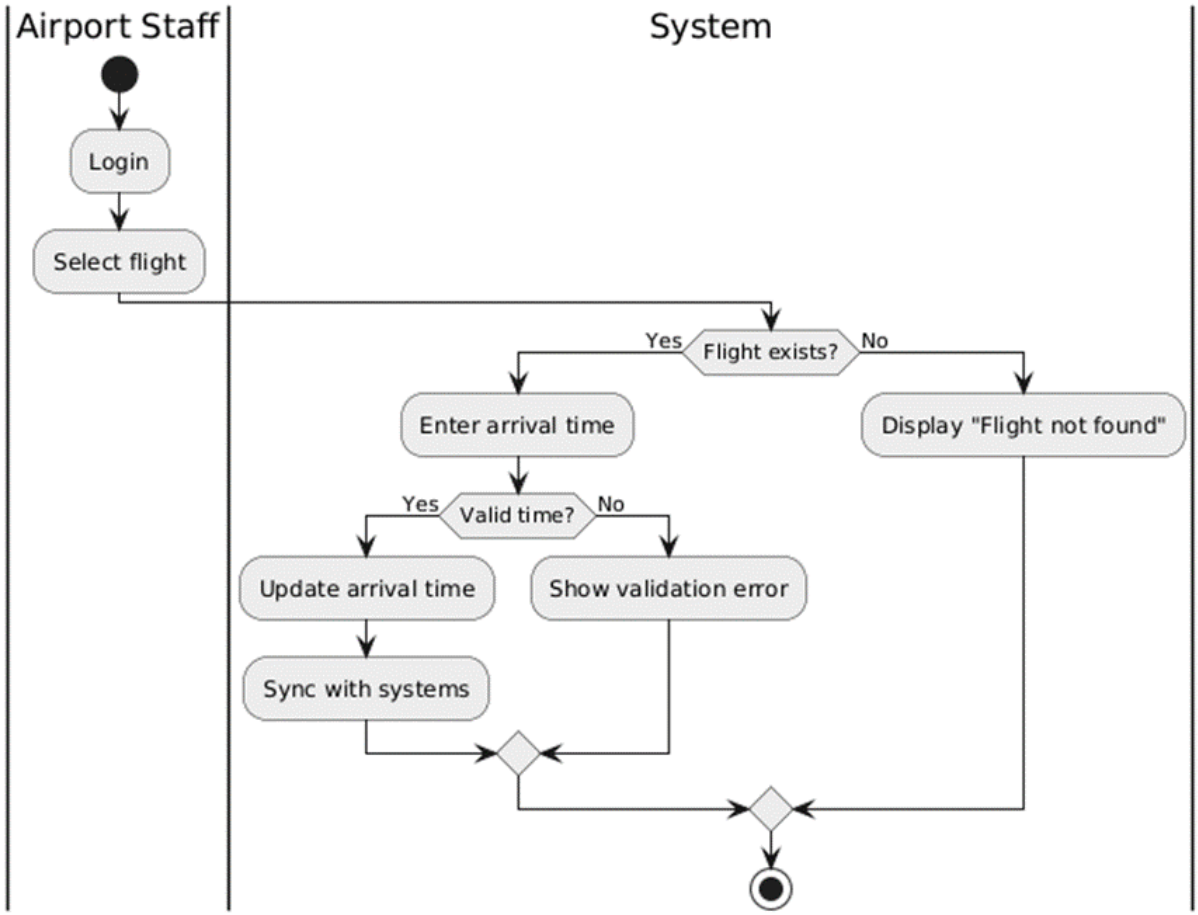
System notifies relevant stakeholders.

Alternative Flow Mapped:

If the flight does not exist, the system displays an error.

If the new time conflicts with runway scheduling, the system rejects the update and prompts correction.

Update arrival time



Core Purpose: *This use case describes how airport staff update the arrival time of a flight.*

Functional Requirements Mapped:

Airport staff updates arrival times.

System validates arrival data.

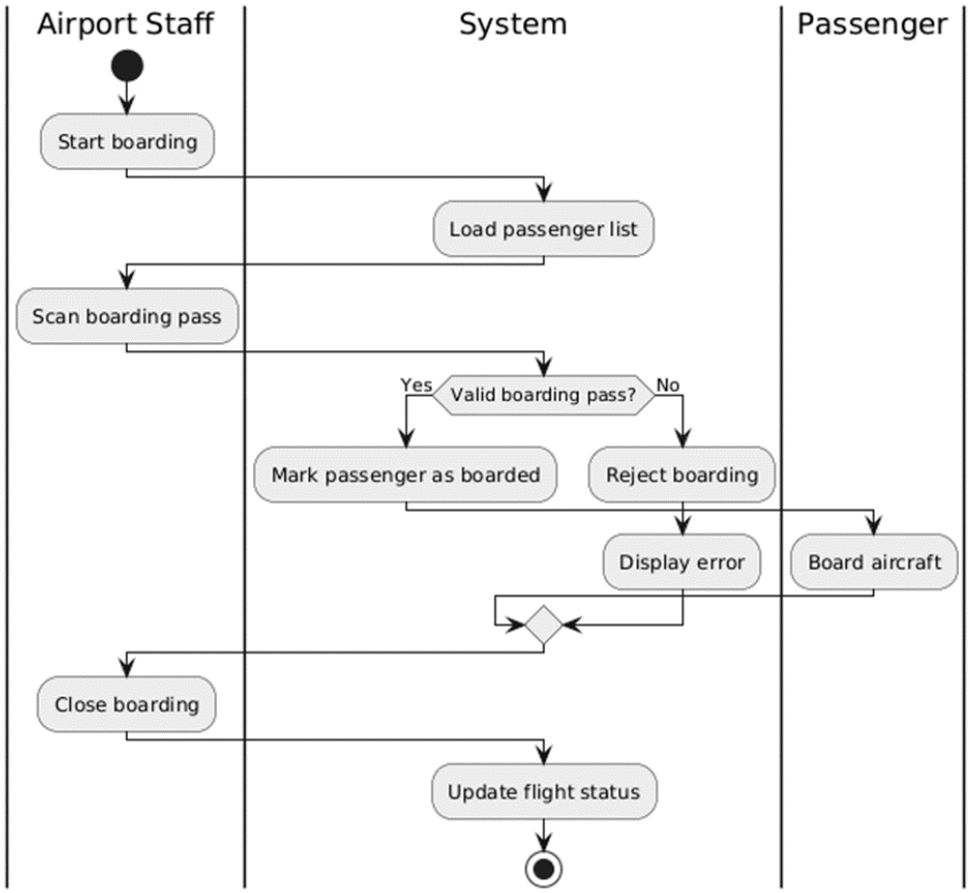
System synchronizes updates with operational displays.

Alternative Flow Mapped:

If the flight does not exist, the system displays an error.

If invalid time data is entered, the system requests correction.

Coordinate Boarding Process



***Core Purpose**: This use case describes how airport staff manage and control the passenger boarding process.*

Functional Requirements Mapped:

Airport staff coordinates boarding processes.

System validates boarding passes.

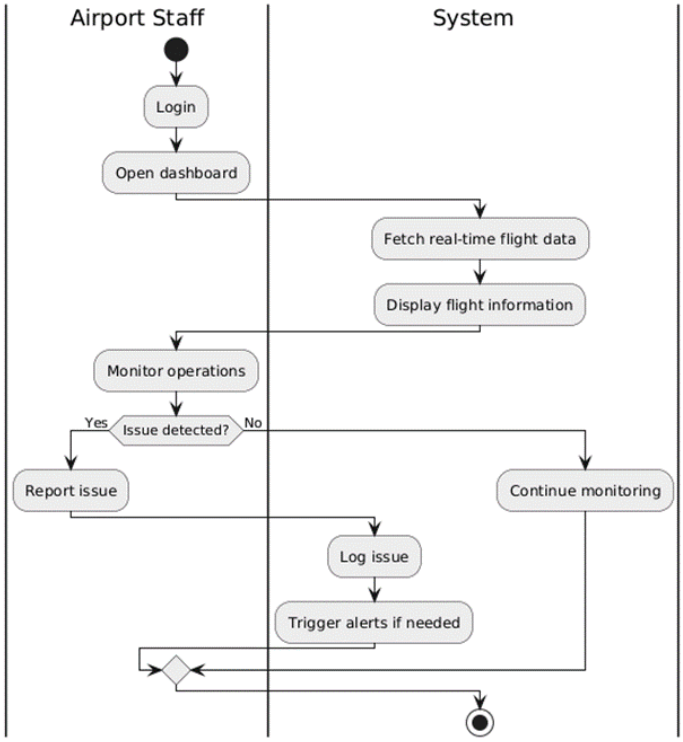
System updates passenger boarding status.

System updates flight status after boarding completion.

Alternative Flow Mapped:

If a boarding pass is invalid, the system denies boarding and alerts staff.

Monitor Flight Operations



Core Purpose*: This use case describes how airport staff monitor real-time flight operations and respond to operational issues.*

Functional Requirements Mapped:

Airport staff monitors flight operations.

System provides real-time operational data.

System logs operational issues.

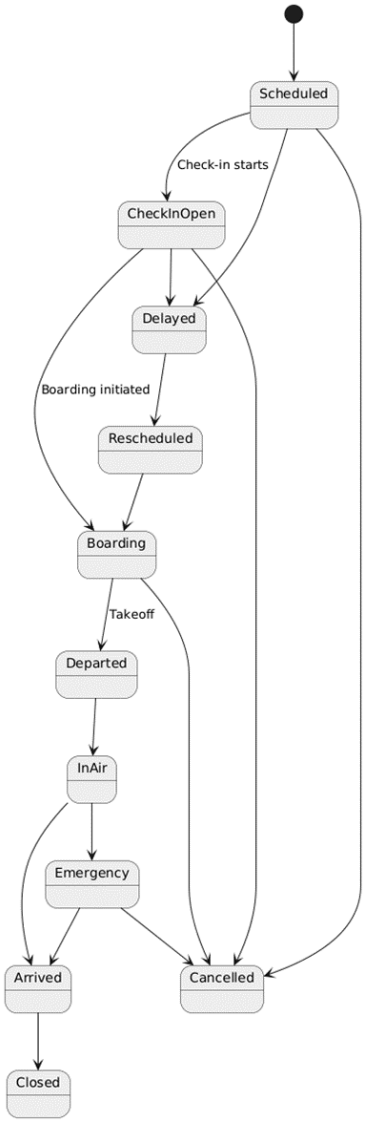
System triggers alerts when necessary.

Alternative Flow Mapped:

If system data retrieval fails, the system displays last known data with a warning.

If no issues are detected, monitoring continues normally.

State Diagram for the whole module



Airport Management System

Security Management Models Documentation

Prepared by:

Ani Velo

Course:

Software Modeling and Design

Date:

26 March 2026

Document Overview

This document presents a structured set of behavioral models for the Airport Management System, specifically focusing on the Security Management System. It translates the defined functional requirements into visual workflows to ensure terminal safety and operational integrity. It includes:

✓ Passenger & Staff Access Activity Diagram (UC-SEC-01, UC-SEC-02)
✓ Contraband & Seizure Management Activity Diagram (UC-SEC-05)
✓ Surveillance & Incident Reporting Activity Diagram (UC-SEC-03, UC-SEC-04)
✓ Emergency Lockdown Activity Diagram (UC-SEC-06)
✓ Security Incident Lifecycle State Diagram

Each model is defined with:

5. ***Mapped Functional Requirements: Linking diagrams to specific UC IDs.***
6. ***Mapped Alternative Flows: Modeling system behavior during errors or threats.***
7. ***Non-functional Requirements: Defining performance and security constraints.***
8. ***Swimlane Actor Interactions: Visualizing the flow between Passengers, Staff, and the System.***

The goal of this section is to clearly model and describe the security system's behavioral workflows in a professional and structured way, serving as a foundation for the system's implementation phase.

Terminal Access and Screening Operation

(UC-SEC-01, UC-SEC-02, UC-SEC-05)

This table covers the physical movement of people and items.

Core Purpose: To manage the secure entry of all individuals into the airport terminal and restricted zones, ensuring that all identities are verified and prohibited items are seized and logged.

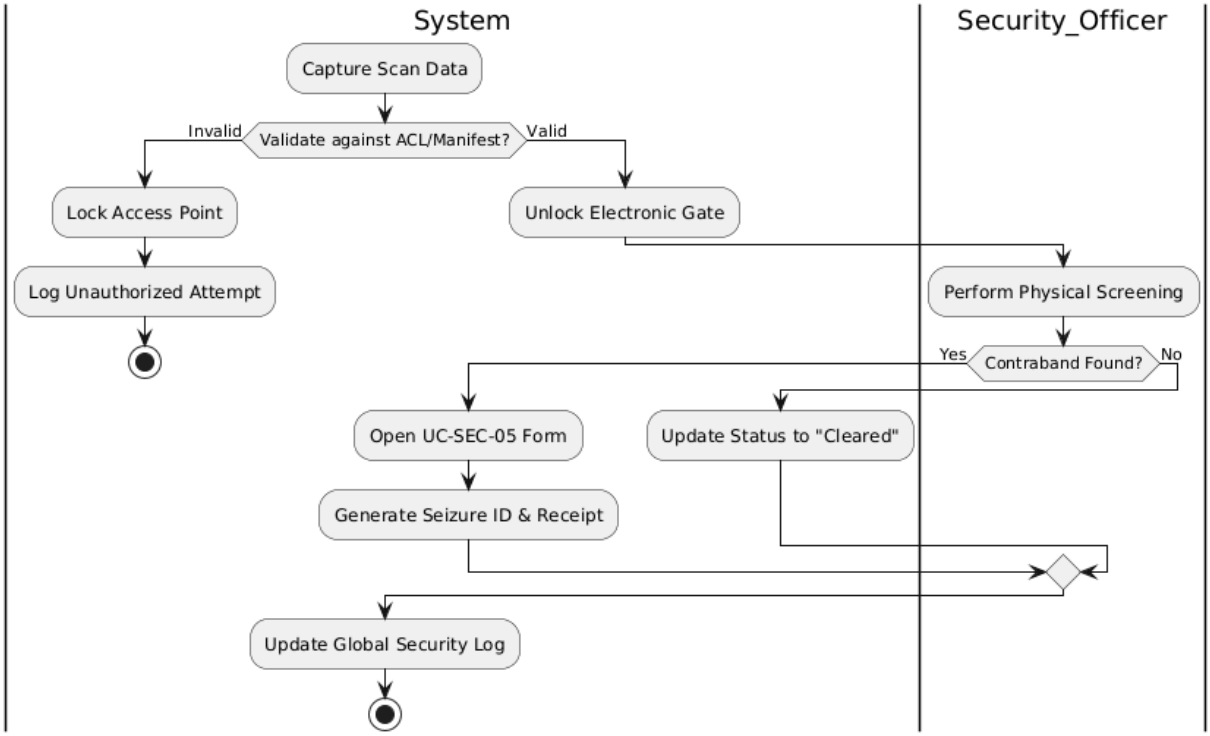
Functional Requirements Mapped:

- *UC-SEC-01*: Passenger Security Screening (Boarding pass & X-ray check.
- *UC-SEC-02*: Restricted Area Access (Staff credential verification).
- *UC-SEC-05*: Log Prohibited Items (Seizure records and receipts).

Alternative Flow Mapped: If credentials are invalid, the system denies entry and logs a warning. If a threat is detected, the system triggers a secondary manual search and logs the event for supervisor review.

Non-functional Requirements:

- *Performance*: Authentication and scanning status must update in < 1 second.
- *Integrity*: Contraband logs must be immutable for legal evidence.



Surveillance, Incident Reporting, and Emergency Response

(UC-SEC-03, UC-SEC-04, UC-SEC-06)

This table covers the digital monitoring and high-level emergency commands.

Core Purpose: To provide 24/7 visual oversight of the airport and establish a rapid response framework for logging incidents and executing facility-wide lockdowns during high-level threats.

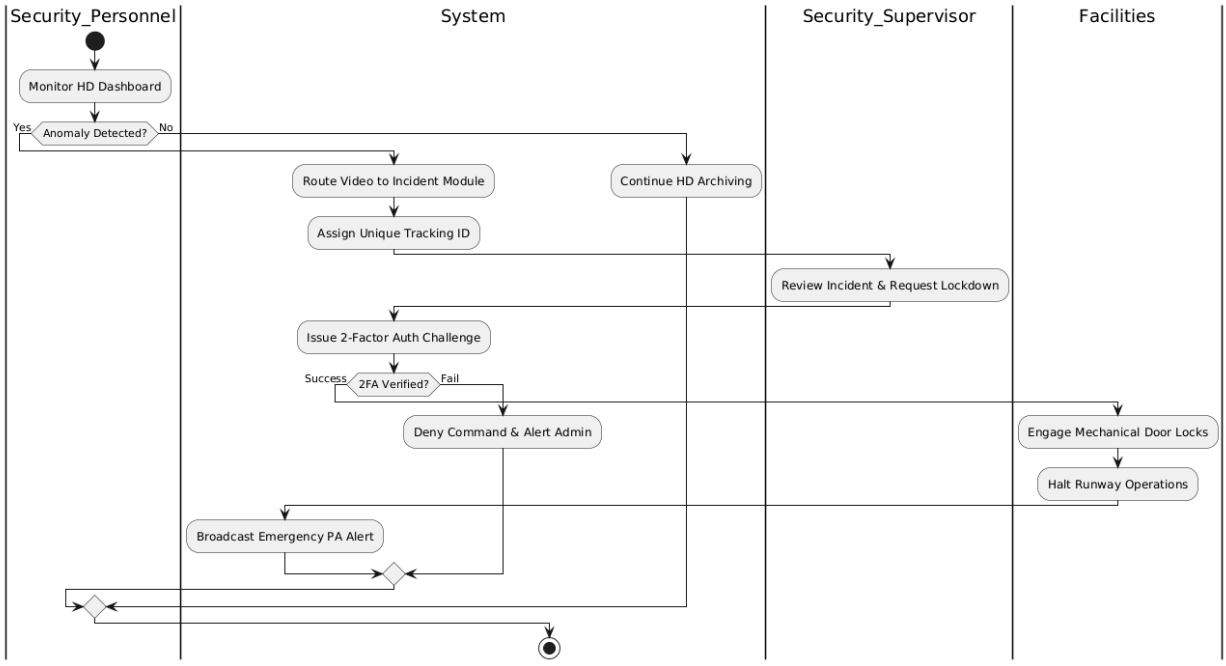
Functional Requirements Mapped:

- *UC-SEC-03*: Report Security Incident (Categorize and prioritize threats).
- *UC-SEC-04*: Monitor Surveillance Feeds (Live HD streaming and 30-day archiving).
- *UC-SEC-06*: Emergency Lockdown (Signal electronic doors and halt flight ops).

Alternative Flow Mapped: If a camera fails, a maintenance ticket is generated. If an incident is "Critical," the system prompts the supervisor to initiate a full lockdown flow.

Non-functional Requirements:

- *Safety:* Lockdown commands must reach hardware in < 2 seconds.
- *Security*: 2-Factor Authentication (2FA) is required for emergency commands



Security Incident Lifecycle

(State Machine)

This table defines the logic for how an incident changes status in the system.

Core Purpose: To track the digital lifecycle and operational status of a security threat from initial discovery through to legal archiving.

State Descriptions:

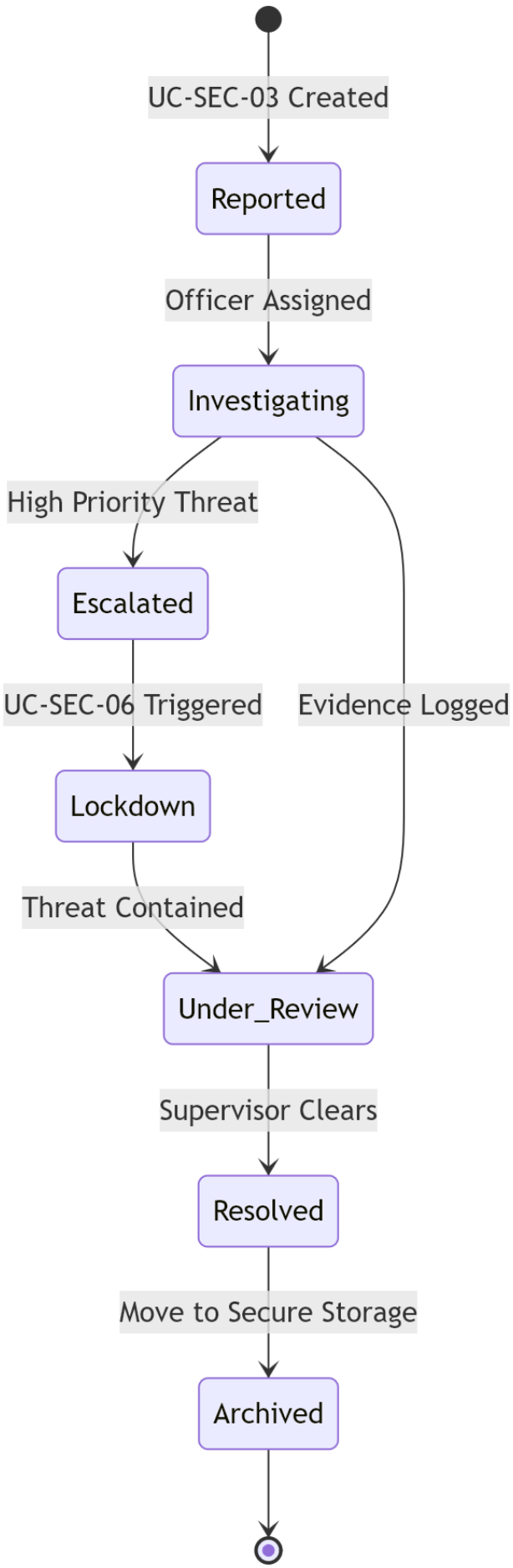
- *Reported*: The initial entry of a threat into the system via UC-SEC-03.
- *Investigating*: An active state where security officers are reviewing surveillance feeds or performing physical checks.
- *Escalated*: A high-alert state triggered when a threat is confirmed as critical.
- *Lockdown*: A restrictive state where UC-SEC-06 is active and facilities are physically secured.
- *Under Review*: The post-incident phase where evidence and logs are verified by a supervisor.
- *Resolved/Archived*: The final state where the threat is closed and data is moved to secure 30-day storage.

T**ransition Logic & Triggers:**

- *Reported* → *Investigating*: Triggered by "Assign Officer" action in the dashboard.
- *Investigating* → *Escalated*: Triggered by "Confirm Threat" button if an anomaly is found.
- *Escalated* → *Lockdown*: Triggered by successful 2-Factor Authentication (2FA) by a supervisor.
- *Under Review* → *Resolved*: Triggered by an administrative digital signature.

Guard Conditions:

- *Authorized* == True: Lockdown cannot be entered without 2FA.
- *All Fields Complete*: An incident cannot move to "Resolved" if the seizure receipt (UC-SEC-05) is missing.


UC-SEC-03 Created
Reported
Officer Assigned
Investigating
High Priority Threat
Escalated
UC-SEC-06 Triggered
Lockdown
Threat Contained
Evidence Logged
Under_Review
Supervisor Clears
Resolved
Move to Secure Storage
Archived

Airport Management System

Airport Resource Management Management Models Documentation

Prepared by:

Ameo Lulaj

Course:

Software Modeling and Design

Date:

26 March 2026

Document Overview

This document presents a structured set of behavioral models for the Airport Management System, specifically focusing on ***Airport Resource Management and Customer Support Management****. It includes:*

✓ *Gate Allocation Activity Diagram (UC-ARM-01)*
✓ *Runway Scheduling Activity Diagram (UC-ARM-02)*
✓ *Staff Allocation Activity Diagram (UC-ARM-03)*
✓ *Equipment Management Activity Diagram (UC-ARM-04)*
✓ *Airport Resource Lifecycle State Diagram (Gate/Asset Lifecycle)*

✓ *Ticket Management Activity Diagram (UC-CSM-01)*
✓ *Live Chat Support Activity Diagram (UC-CSM-02)*
✓ *Support Ticket Lifecycle State Diagram*

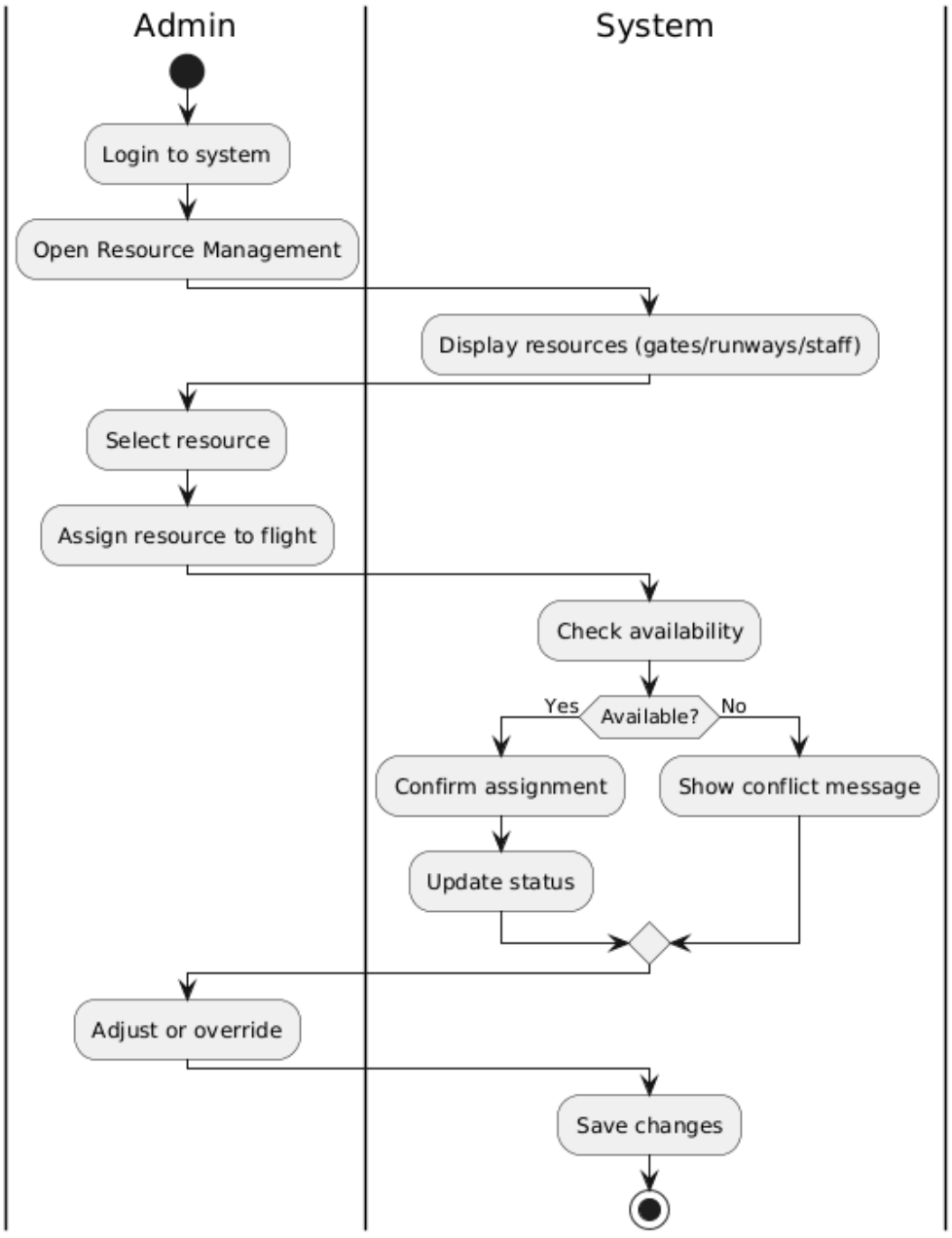
✓ *Each model is defined with:*

1. *Mapped Functional Requirements*
2. *Mapped Alternative Flows*
3. *Non-functional Requirements*
4. *Swimlane Actor Interactions*

The goal of this document is to clearly model and describe the system's behavioral workflows in a professional and structured way.

Core Purpose:

To efficiently manage and allocate airport resources such as gates, runways, staff, and equipment to ensure smooth airport operations, avoid conflicts, and optimize utilization.



ACTIVITY DIAGRAM OF

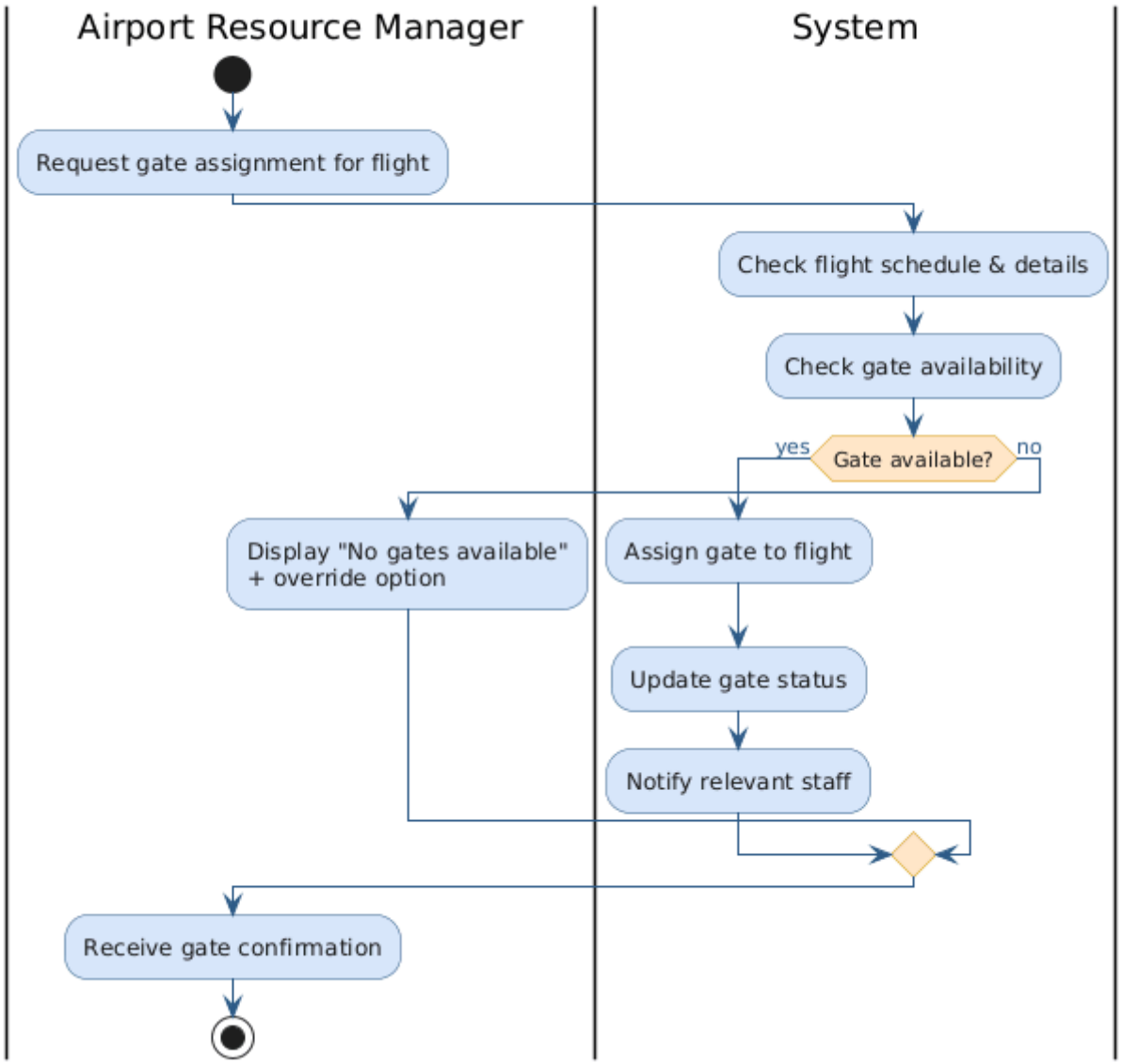
Functional Requirements Mapped:

- **UC-ARM-01 (Gate Allocation Management):**
  Assign gates to flights, check availability, and prevent scheduling conflicts.
- **UC-ARM-02 (Runway Scheduling):**
  Schedule runways for takeoff and landing with conflict detection and priority handling.
- **UC-ARM-03 (Staff Allocation):**
  Assign staff to tasks and shifts based on availability.
- **UC-ARM-04 (Equipment Management):**
  Track equipment status and manage maintenance activities.

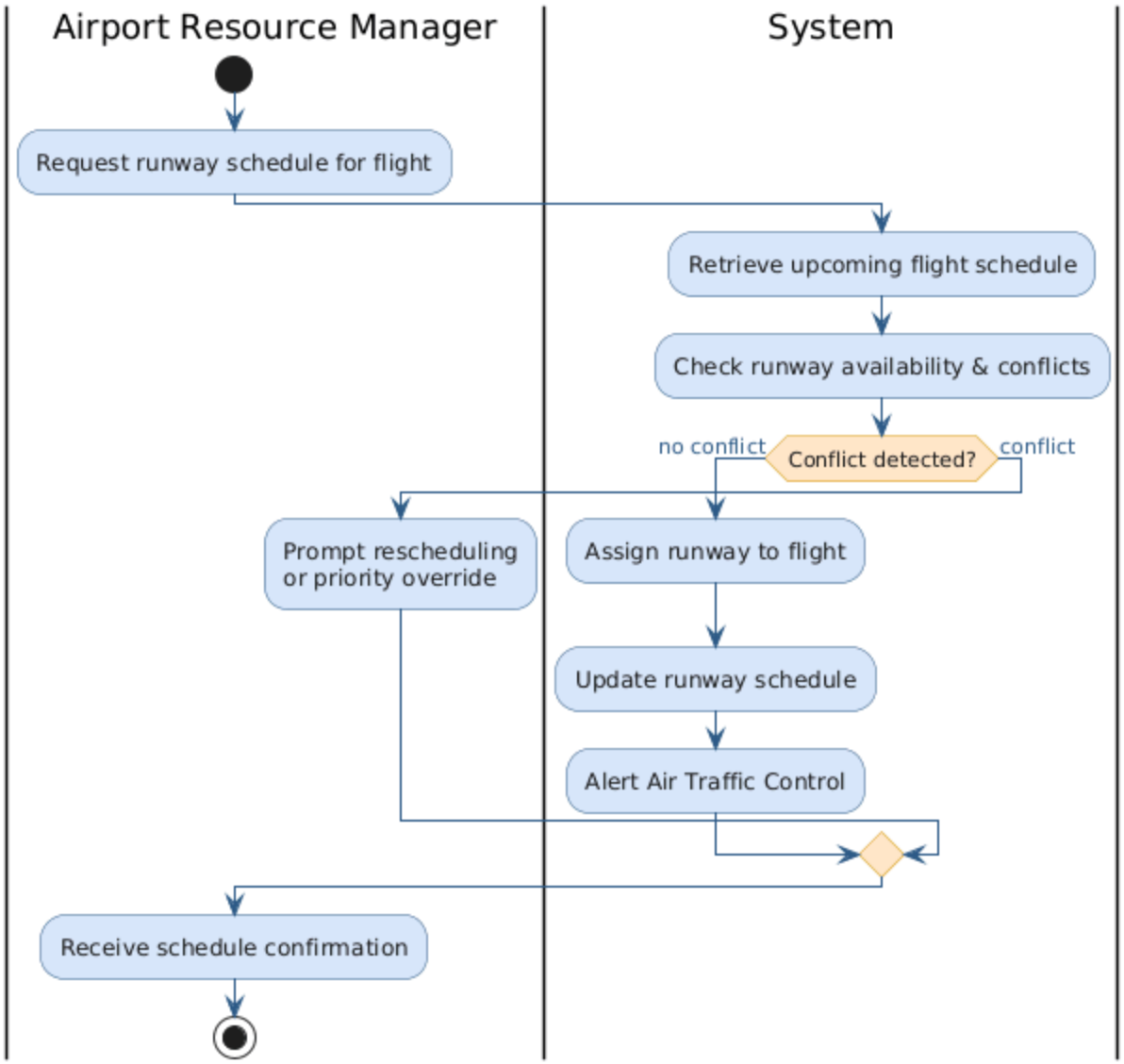
Alternative Flow Mapped:

- **Gate Allocation (UC-ARM-01):**
  If no gates are available, the system displays an alert and allows manual override.
- **Runway Scheduling (UC-ARM-02):**
  If a scheduling conflict occurs, the system prompts rescheduling. Emergency flights are prioritized.
- **Staff Allocation (UC-ARM-03):**
  If selected staff is unavailable, the system suggests alternative staff.
- **Equipment Management (UC-ARM-04):**
  If equipment fails, the system triggers a maintenance alert and updates status.

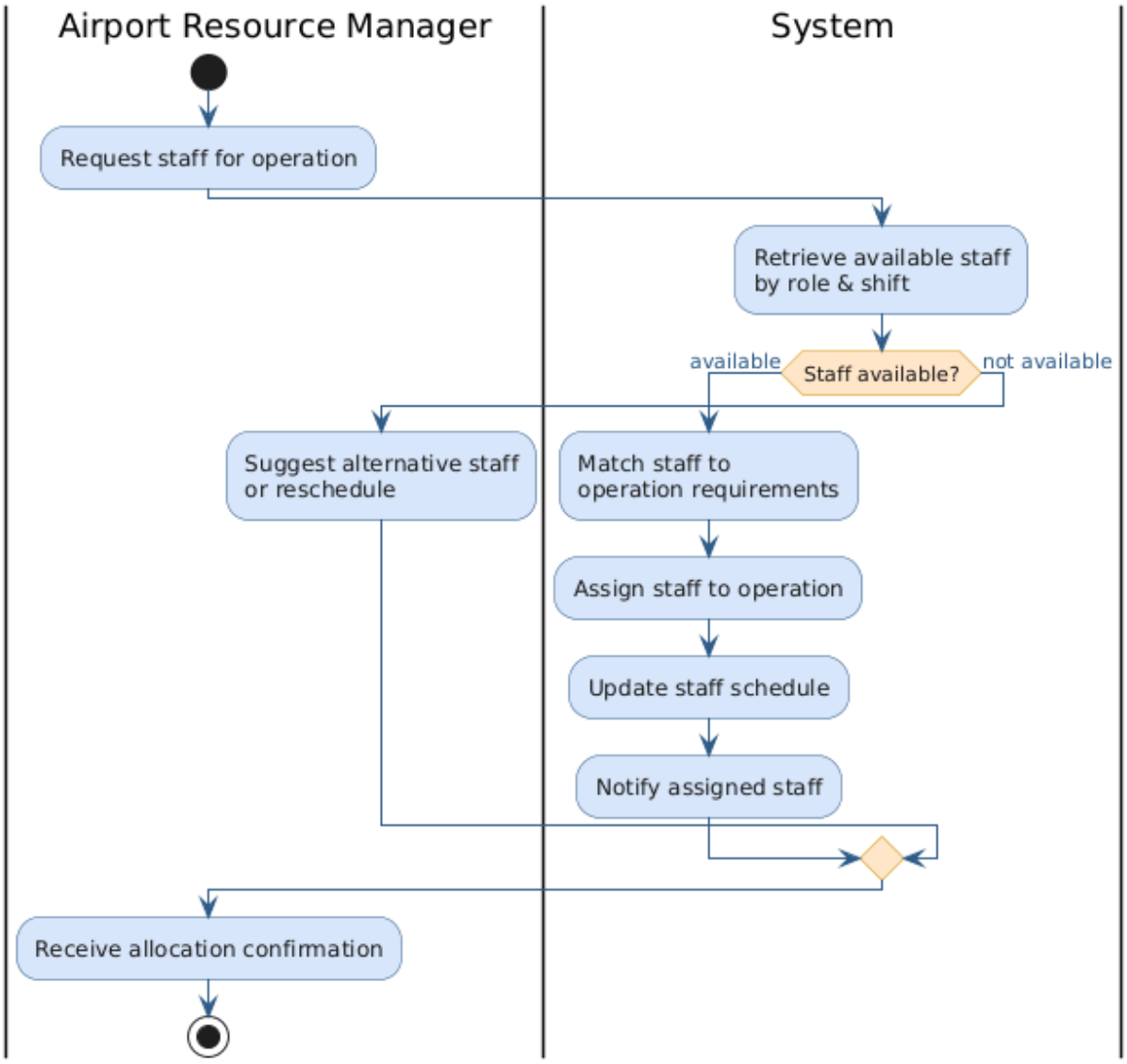
UC-ARM-01: Gate Allocation — Activity Diagram



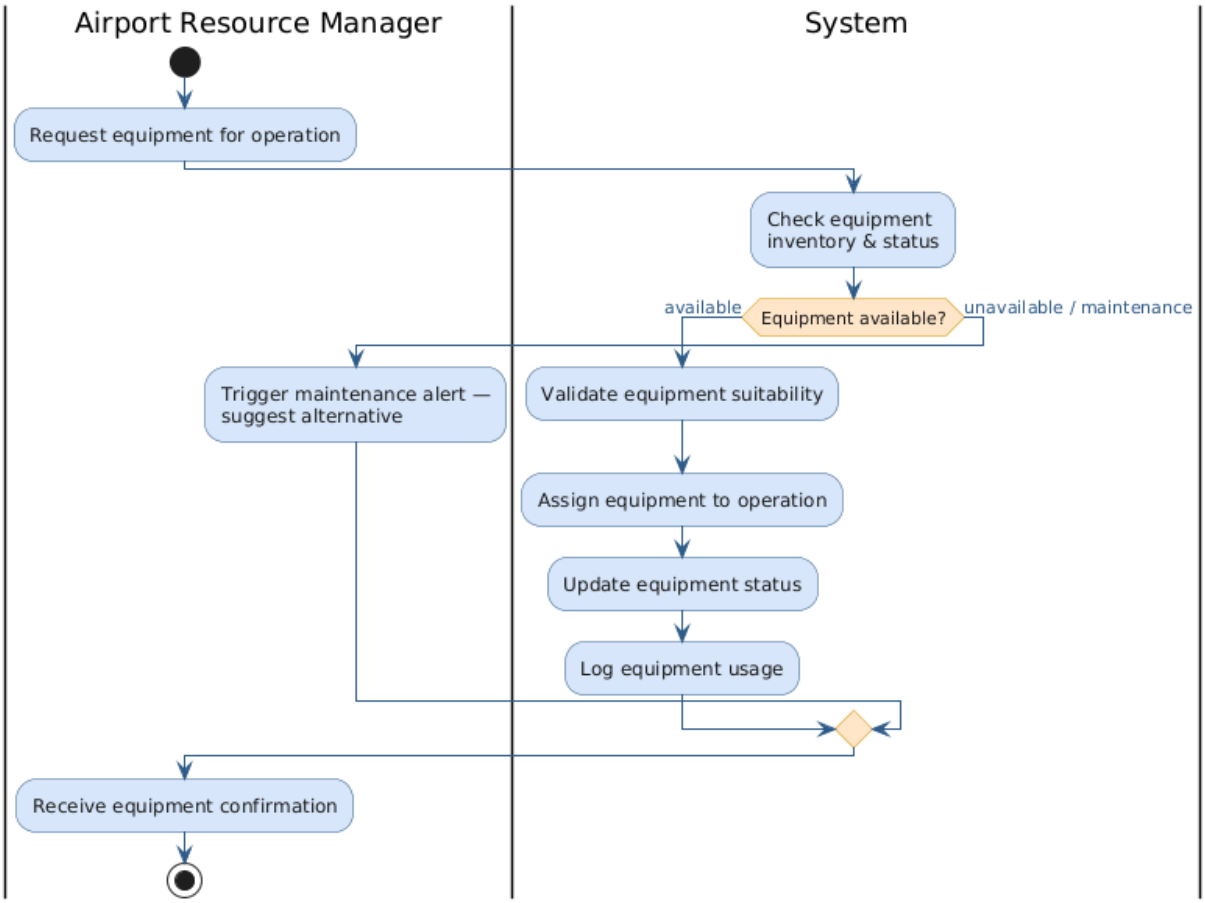
UC-ARM-02: Runway Scheduling — Activity Diagram



UC-ARM-03: Staff Allocation — Activity Diagram



UC-ARM-04: Equipment Management — Activity Diagram



Airport Management System

Finance Office and Human Resources Use Cases Documentation

Prepared by:

Andri Mucllari

Course:

Software Modeling and Design

Date:

27 March 2026

Document Overview

This document presents a structured set of **Finance Office and Human Resources models** for the Airport Management System. It includes:

✔ Processing Airport Service Payments (UC_50)
✔ Generating Airport Service Invoices (UC_51)
✔ Monitoring Financial Transactions (UC_52)
✔ Managing Airport Staff Records (UC_53)
✔ Managing Staff Attendance (UC_54)
✔ Processing Payroll (UC_55)

Each use case is defined with:

- **Mapped Functional Requirements:** Linking system functionality to specific UC IDs.
- **Mapped Alternative Flows:** Modeling system behavior during errors or exceptional conditions.
- **Non-functional Requirements:** Defining performance, security, and reliability constraints.
- **Swimlane Actor Interactions:** Visualizing the interaction between Passengers, Finance Staff, HR Staff, Employees, and the System.

The goal of this document is to clearly model and describe **financial and human resource system operations** in a professional and structured manner.

Processing Airport Service Payments (UC_50)

This use case describes how passengers pay for airport services such as parking, lounge access, or baggage handling through the system.

• Functional Requirements Mapped:

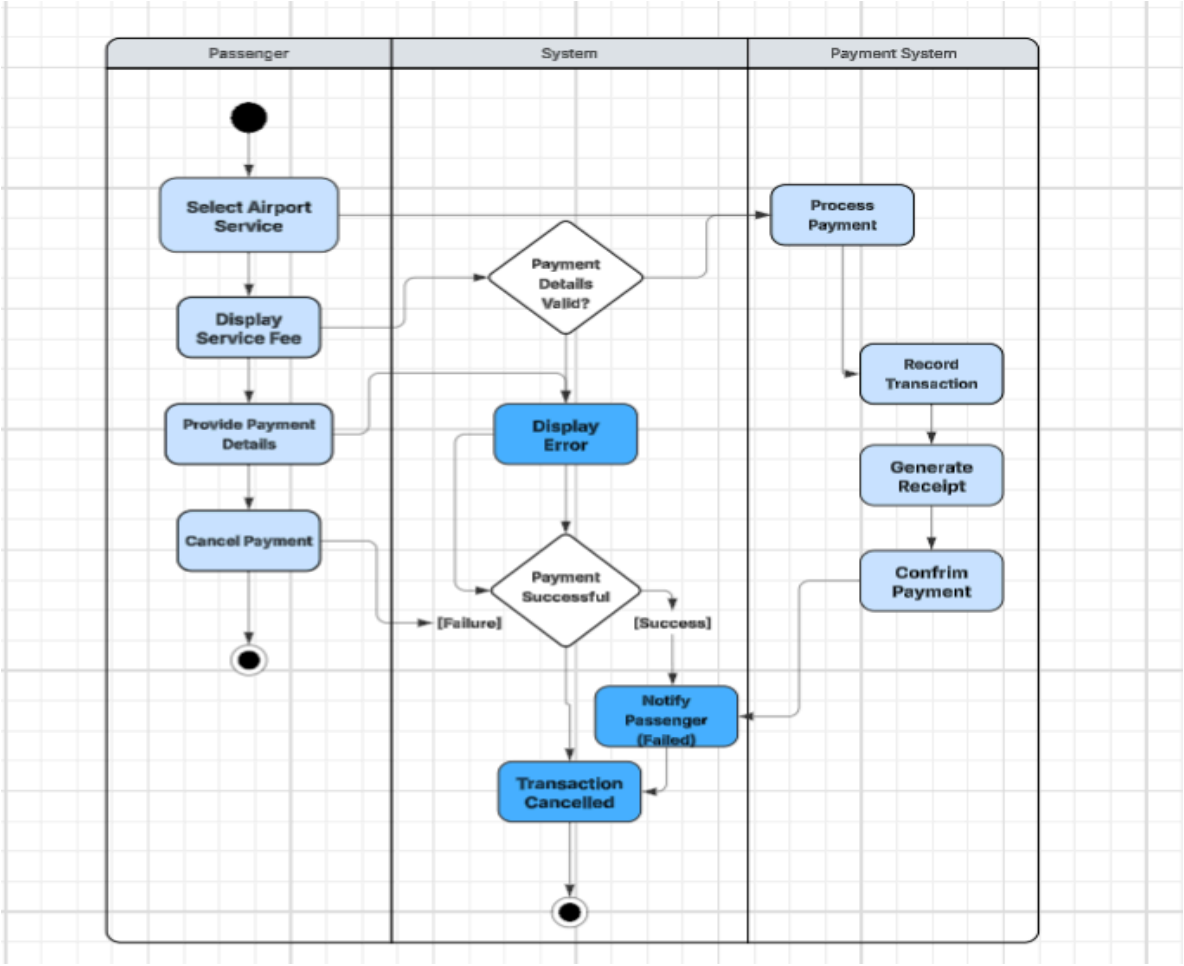
- *Passenger selects an airport service.*
- *System displays the corresponding service fee.*
- *Passenger provides payment details.*
- *System validates the payment information.*
- *System processes the payment through the payment system.*
- *System records the transaction in the database.*
- *System generates a receipt.*
- *System confirms successful payment to the passenger.*

• Alternative Flow Mapped:

If invalid payment details are entered, the system displays an error message. If the payment fails, the system notifies the passenger. If a system error occurs, the transaction is cancelled. If the passenger cancels the process, no payment is processed.

• Non-Functional Requirements Mapped:

Payment processing must be secure, transactions must be completed quickly, all payments must be logged, and user data privacy must be ensured.



Generating Airport Service Invoices (UC_51)

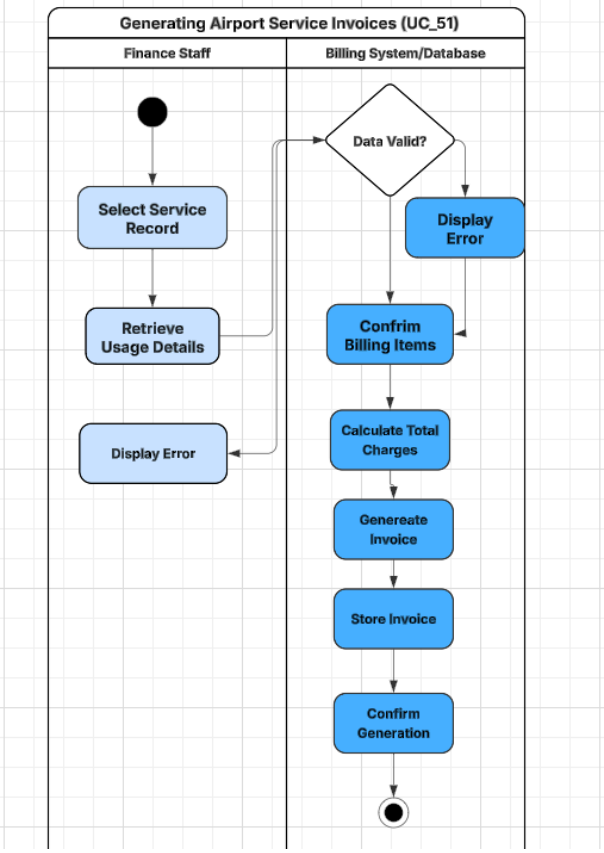
This use case describes how finance staff generate invoices for airport services provided to external parties such as aircraft parking and ground handling.

• Functional Requirements Mapped:

- Finance staff accesses the billing or invoice management module.
- System displays available service records.
- Finance staff selects a service record.
- System retrieves and displays usage details.
- Finance staff confirms billing items.
- System calculates total charges.
- System generates the invoice.
- System stores the invoice in the database.
- System confirms successful invoice generation.

• Alternative Flow Mapped:

If required data is missing, the system displays an error message. If invalid input is provided, the system prompts for correction.



Monitoring Financial Transactions (UC_52)

This use case describes how financial activities within the airport are monitored and reviewed by authorized users such as finance staff or system administrators.

• Functional Requirements Mapped:

- User logs into the system.
- User accesses the financial monitoring module.
- System retrieves and displays transaction records.
- User applies filters to refine transaction data.
- System updates and displays filtered results.
- User reviews the financial data.

• Alternative Flow Mapped:

If no financial data is available, the system notifies the user. If a system error occurs, the system is unable to retrieve transaction data and displays an error message.

Managing Airport Staff Records (UC_53)

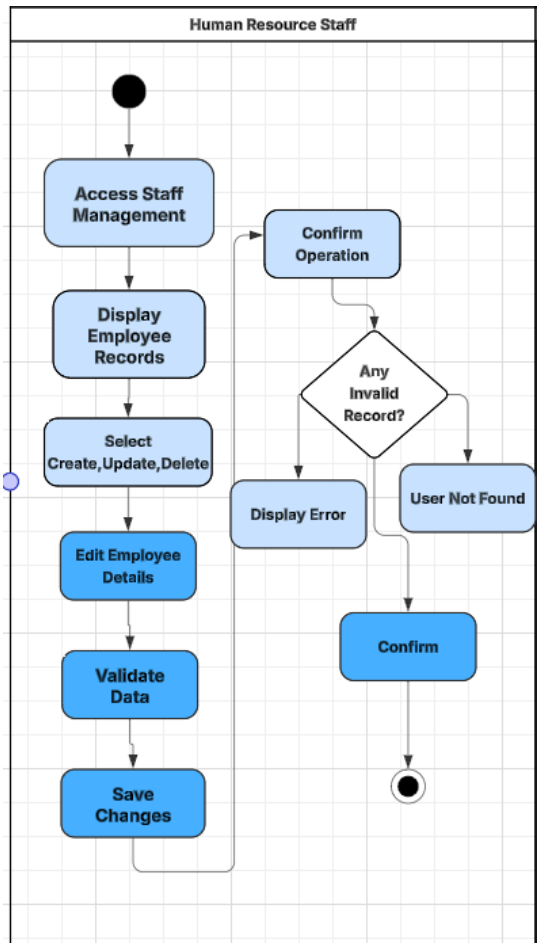
This use case describes how Human Resource (HR) staff manage records of airport employees, including security staff, ground crew, and administrative personnel.

• Functional Requirements Mapped:

- HR staff logs into the system.
- HR accesses the staff management module.
- System displays existing employee records.
- HR selects an action (create, update, or delete).
- HR enters new employee details or modifies existing information.
- System validates the entered data.
- System saves changes in the database.
- System confirms successful operation.

• Alternative Flow Mapped:

If invalid data is entered, the system displays an error message. If a selected record does not exist, the system notifies the user. If a system failure occurs, the operation is cancelled and no changes are saved.



Managing Staff Attendance (UC_54)

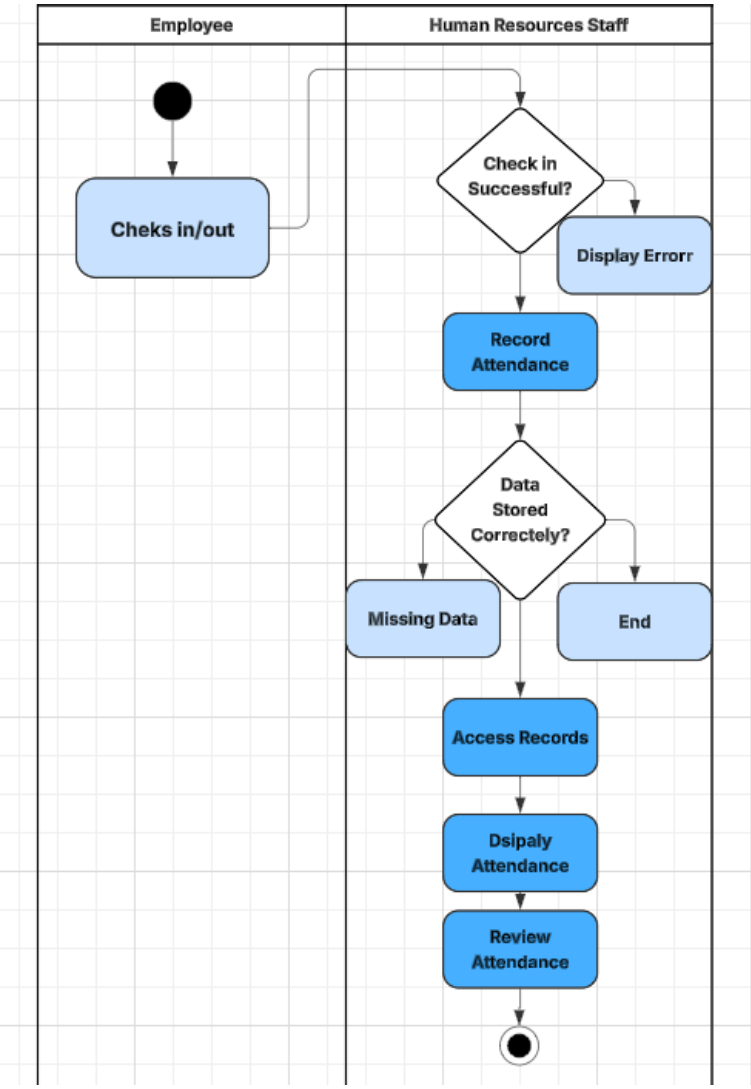
This use case describes how airport staff attendance is recorded and monitored by the system and reviewed by HR staff.

• Functional Requirements Mapped:

- Employee checks in or checks out through the system.
- System records the attendance time.
- HR staff accesses the attendance management module.
- System displays attendance records.
- HR reviews attendance logs.

• Alternative Flow Mapped:

If the check-in or check-out process fails, the system displays an error message. If attendance data is missing, the system notifies HR staff.



Processing Payroll (UC_55)

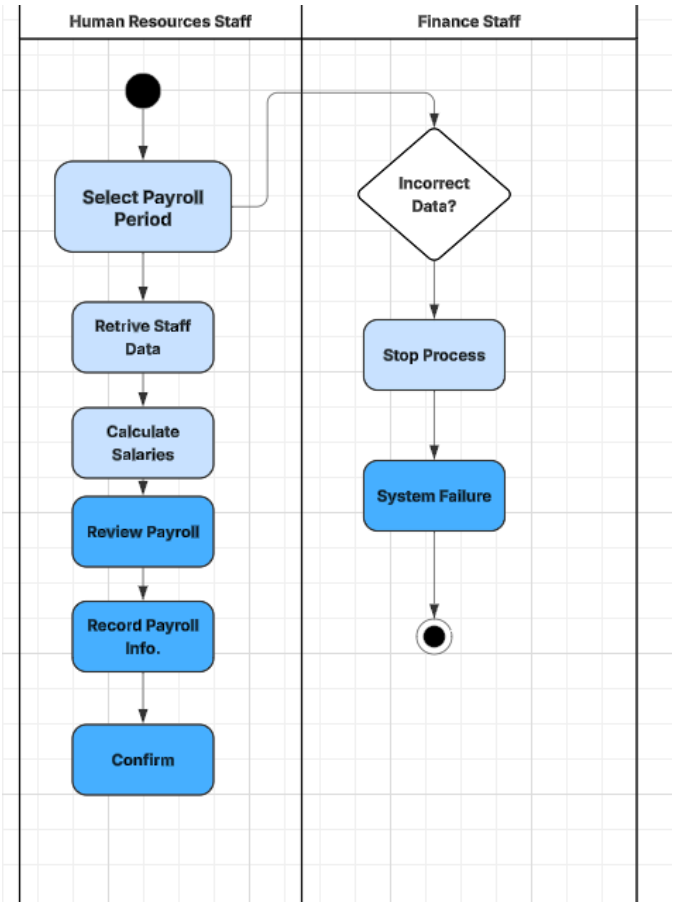
This use case describes how salaries for airport staff are calculated and processed based on employee and attendance data.

• Functional Requirements Mapped:

- HR or Finance staff accesses the payroll module.
- HR selects the payroll period.
- System retrieves employee and attendance data.
- System calculates salaries, including overtime and deductions.
- HR reviews payroll details.
- System processes salary payments.
- System records payroll information in the database.
- System confirms successful completion of payroll processing.

• Alternative Flow Mapped:

If required data is missing, the system displays an error message. If a calculation error occurs, the process is stopped. If a system failure occurs, payroll processing is cancelled and no payments are made.



Airport Management System

Customer Support & Human Resource Management Models Documentation

Prepared by:

Ajla Alcani

Course:

Software Modeling and Design

Date:

26 March 2026

Document Overview

This document presents a structured set of behavioral models for the Airport Management System. It includes:

✔ *Ticket Management Activity Diagram*

✔ *Live Chat Support Activity Diagram*

✔ *Support Ticket Lifecycle State Diagram*

✔ *Manage Airport Staff Records Activity Diagram*

✔ *Manage Staff Attendance Activity Diagram*

✔ *Process Payroll Activity Diagram*

✔ *Each model is defined with:*

1. Mapped Functional Requirements

2. Mapped Alternative Flows

3. Non-functional Requirements

4. Swimlane Actor Interactions

The goal of this document is to clearly model and describe the system's behavioral workflows in a professional and structured way.

The following section presents the visual behavioral models for the Airport Management System (AMS), specifically focusing on the Customer Support Management and Human Resource Management modules. These Activity and State Machine diagrams translate the defined functional and non-functional requirements from the Software Requirements Specification (SRS) into structured workflows. By mapping each diagram directly to its corresponding

use case—including both main and alternative flows—this section demonstrates how the system design logically fulfills the expected operational capabilities and serves as a foundation for the implementation phase.

1. Customer Support Management Diagrams

Activity Diagram 1: Ticket Management (UC-CSM-01)

• Core Purpose: This use case describes how passengers submit and track support queries using a ticketing system.

• Functional Requirements Mapped:

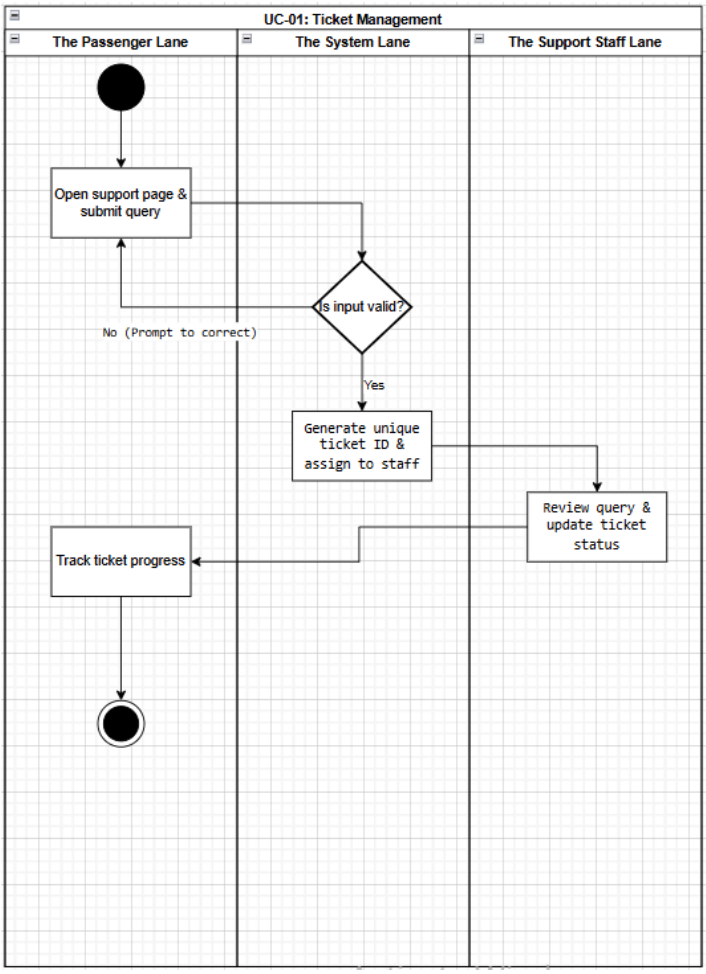
• Customer support agent tracks service requests.

• Passenger submits query via form.

• System generates a unique ticket ID and assigns it to support staff.

• Staff updates ticket status and passenger tracks ticket progress.

• Alternative Flow Mapped: If invalid input is provided, the system prompts the user to correct details.



Activity Diagram 2: Live Chat Support (UC-CSM-02)

• *Core Purpose:* This use case describes how passengers receive real-time assistance through chat support.

• *Functional Requirements Mapped:*

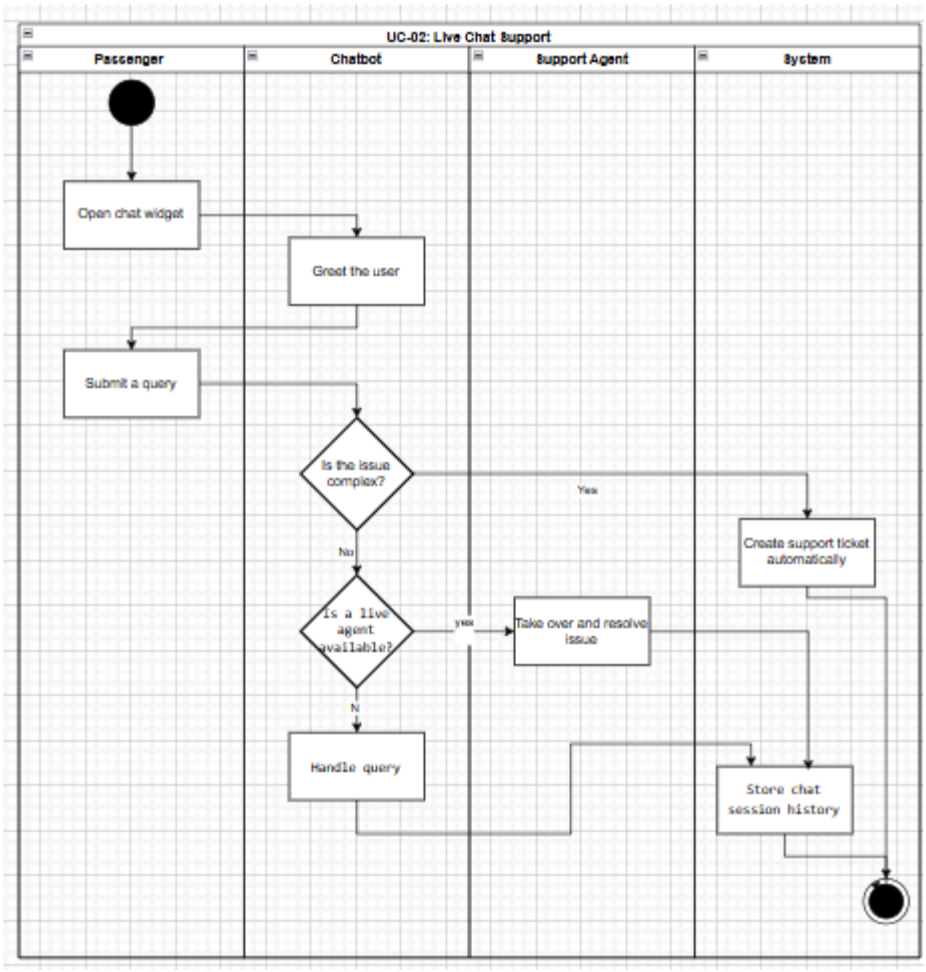
• Customer support agent responds to inquiries.

• Passenger opens chat widget and submits a query.

• Chatbot greets the user and either responds or transfers to a live agent.

• Agent resolves the issue and the chat session is stored.

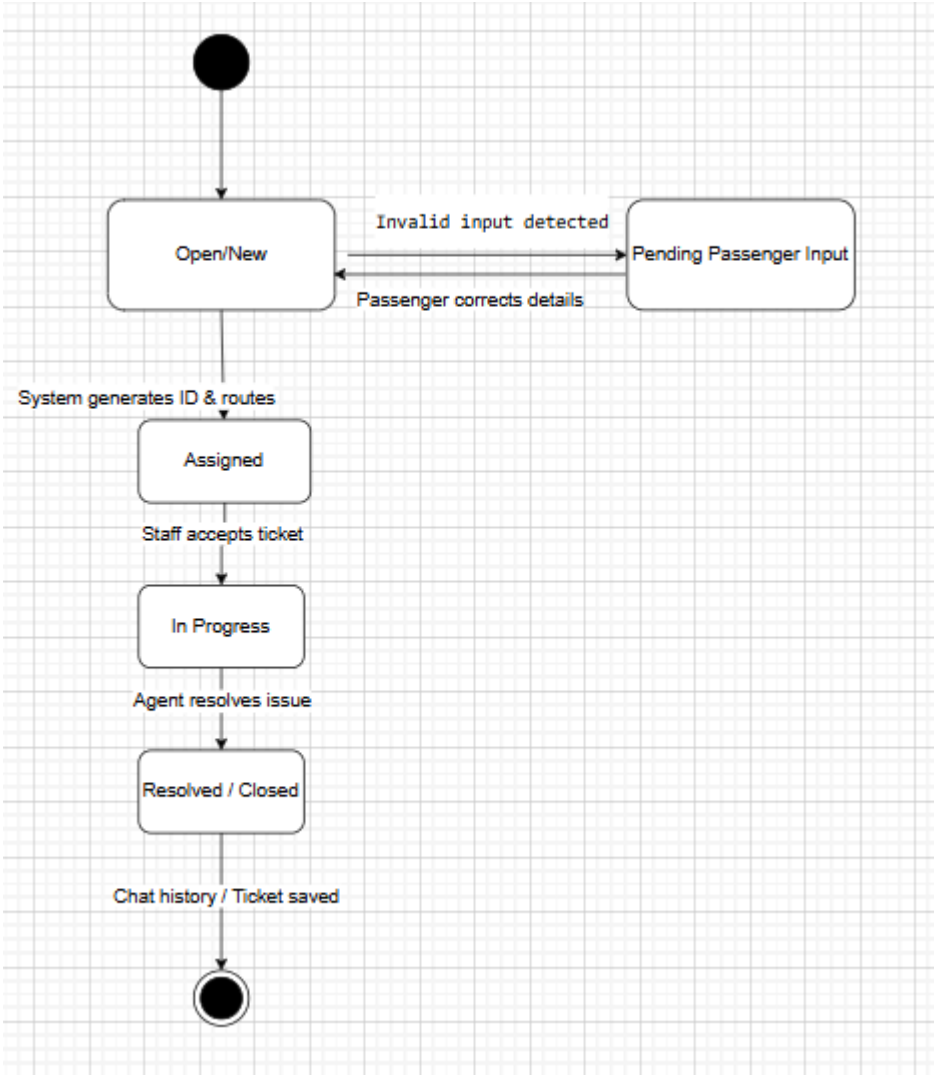
• Alternative Flow Mapped: If the issue is complex, the system creates a support ticket automatically.


UC-02: Live Chat Support
Passenger
Chatbot
Support Agent
System
Open chat widget
Greet the user
Submit a query
Is the issue complex?
Yes
No
Create support ticket automatically
Is a live agent available?
yes
Take over and resolve issue
N
Handle query
Store chat session history

State Diagram: Support Ticket Lifecycle

• *Core Purpose*: To model the exact states a ticket passes through from creation to resolution.

• *Requirements Mapped:* Ensures a ticket is successfully created, tracked, and managed, and that the query is either resolved or escalated.



2. Human Resource Management Diagrams

Activity Diagram 3: Manage Airport Staff Records (UC_53)

• Core Purpose: This use case describes how HR manages records of airport employees.

• *Functional Requirements Mapped:*

• HR staff registers employee records and assigns staff roles.

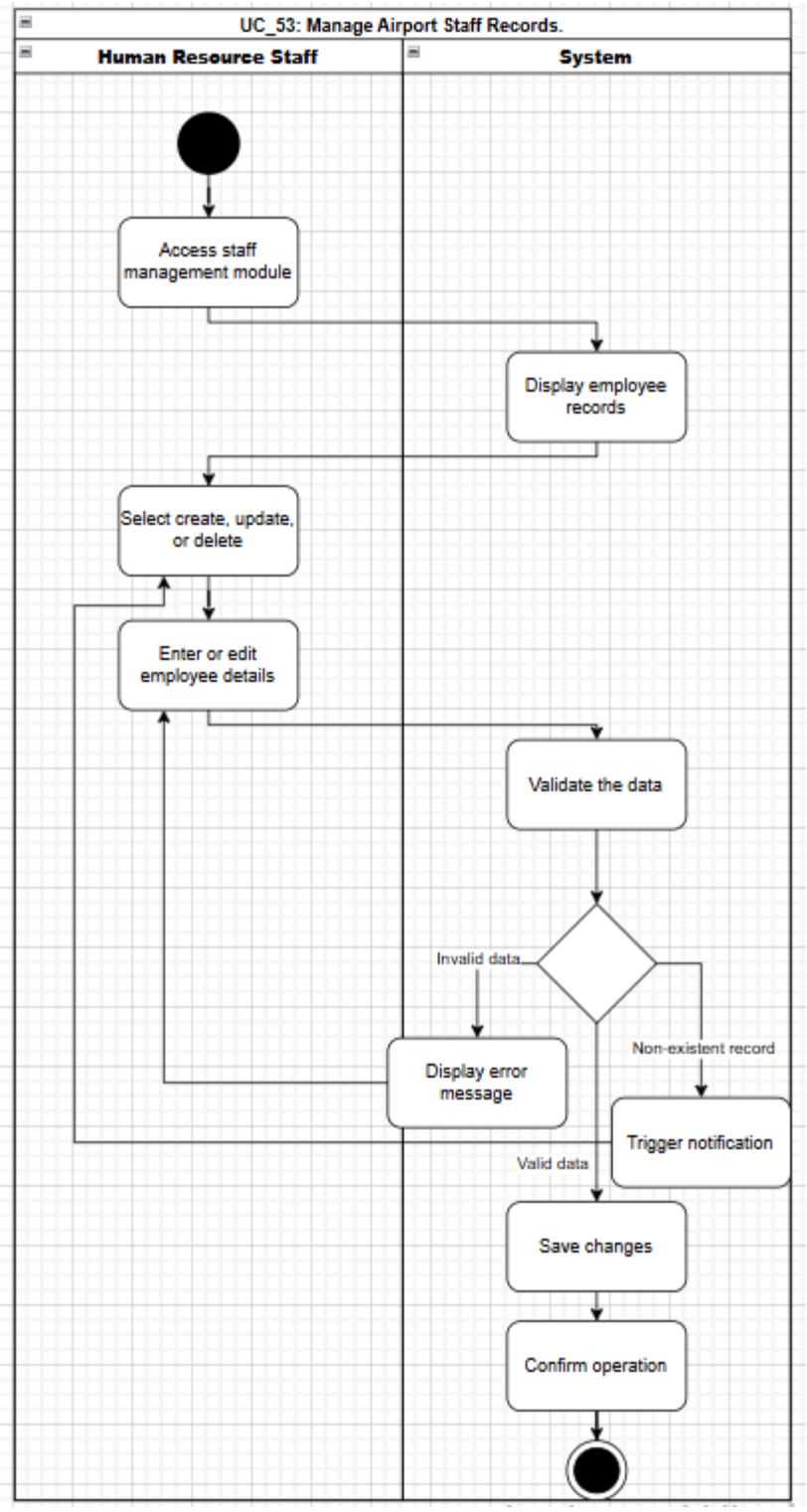
• HR accesses the module and selects to create, update, or delete.

• HR enters or edits employee details.

• System validates the data, saves changes, and confirms the operation.

• *Alternative Flow Mapped:* Invalid data triggers an error message, and a non-existent record triggers a notification.

• *Non-Functional Requirements Mapped:* Employee data must be secure, role-based access must be enforced, and all changes must be logged.


UC_53: Manage Airport Staff Records.
Human Resource Staff
System
Access staff management module
Display employee records
Select create, update, or delete
Enter or edit employee details
Validate the data
Invalid data
Non-existent record
Display error message
Trigger notification
Valid data
Save changes
Confirm operation

Activity Diagram 4: Manage Staff Attendance (UC_54)

• *Core Purpose:* This use case describes how airport staff attendance is recorded and monitored.

• *Functional Requirements Mapped:*

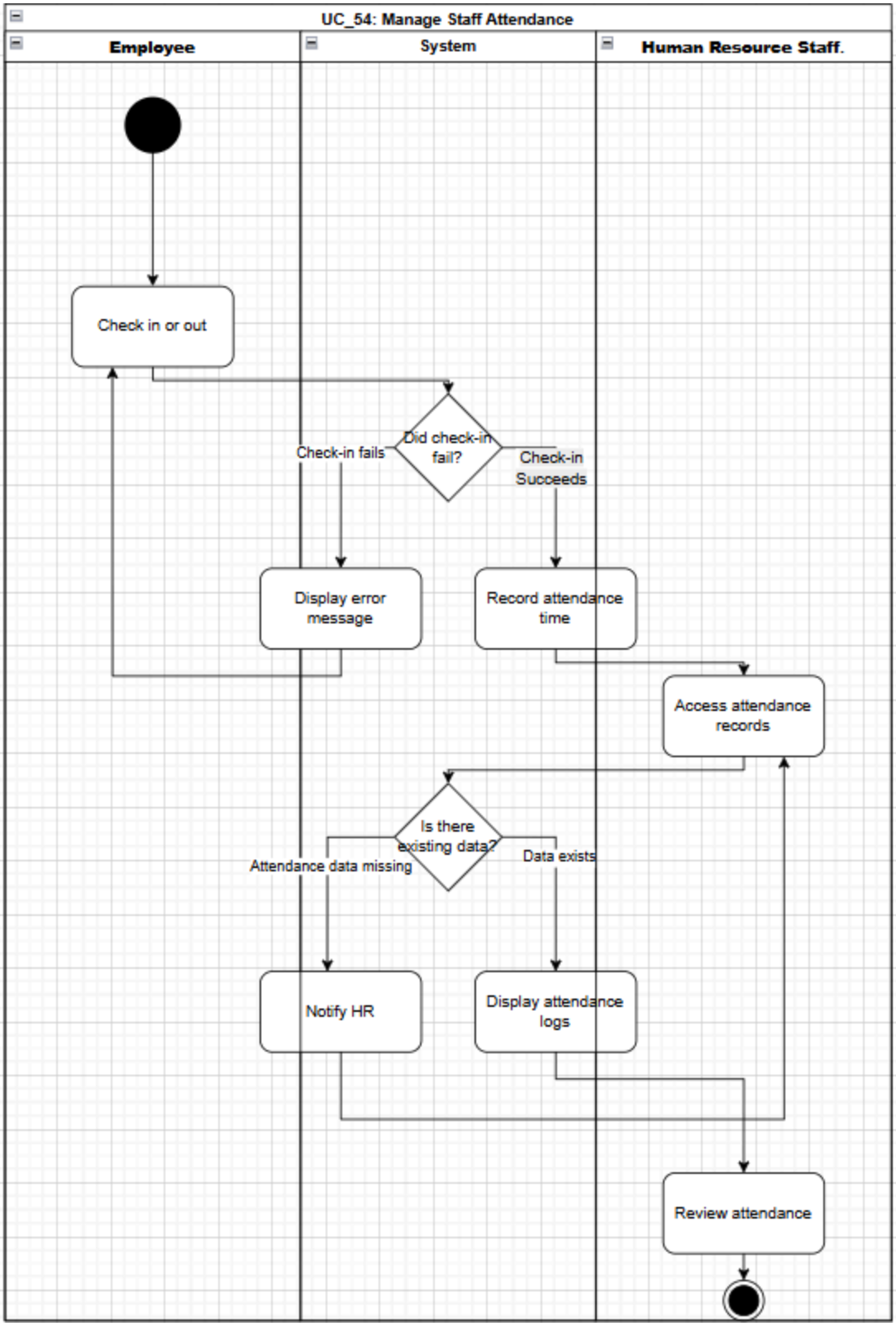
• HR staff manages employee schedules.

• Employee checks in/out and the system records the attendance time.

• HR accesses the records, the system displays the attendance logs, and HR reviews the attendance.

• *Alternative Flow Mapped:* If check-in fails, the system shows an error; if attendance data is missing, the system notifies HR.

• *Non-Functional Requirements Mapped:* System must ensure accurate time tracking, data must be secure, and the system must be reliable.


UC_54: Manage Staff Attendance
Employee
System
Human Resource Staff.
Check in or out
Did check-in fail?
Check-in fails
Check-in Succeeds
Display error message
Record attendance time
Access attendance records
Is there existing data?
Attendance data missing
Data exists
Notify HR
Display attendance logs
Review attendance

Activity Diagram 5: Process Payroll (UC_55)

• *Core Purpose:* This use case describes how salaries for airport staff are calculated and processed.

• *Functional Requirements Mapped:*

• HR selects the payroll period and the system retrieves staff data.

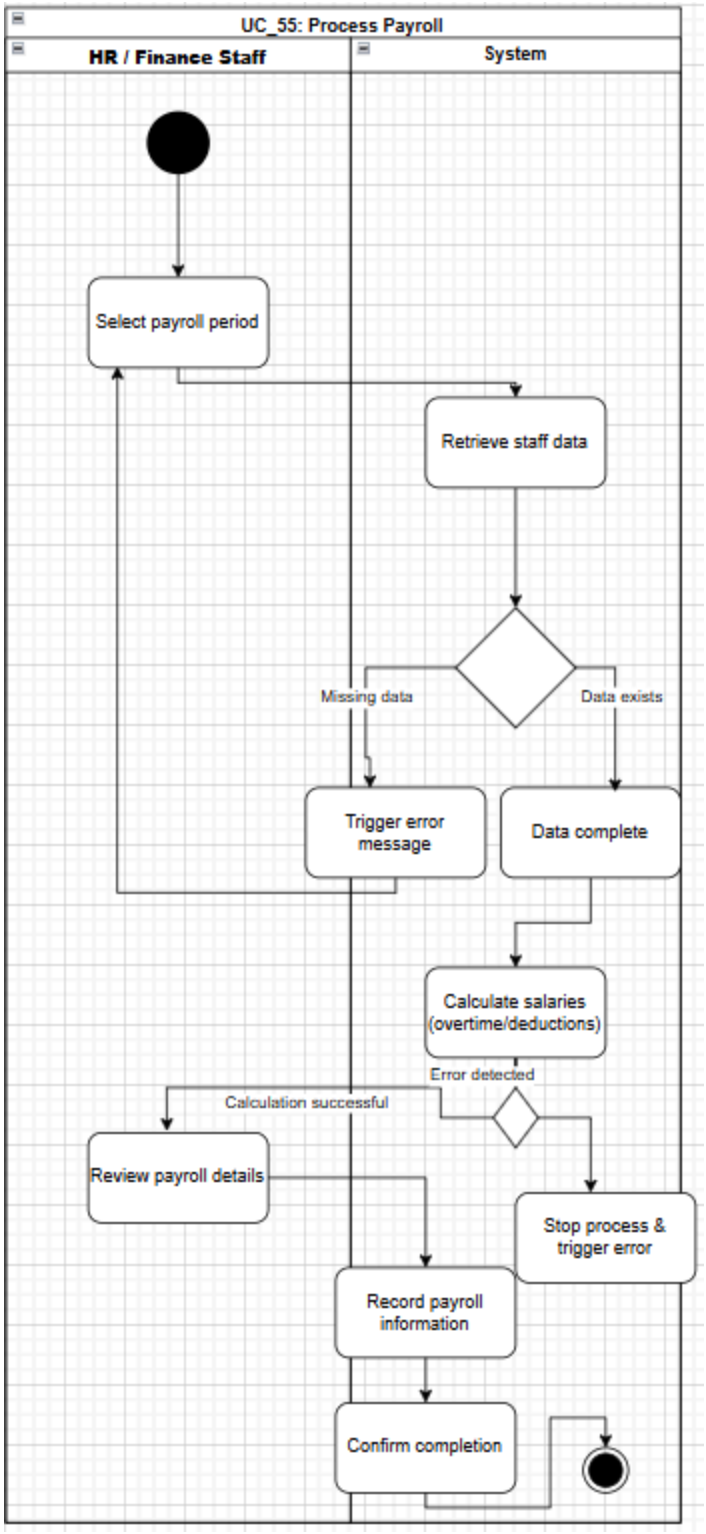
• System calculates salaries, including overtime and deductions.

• HR reviews payroll details.

• System processes salary payments, records payroll information, and confirms completion.

• *Alternative Flow Mapped:* Missing data triggers an error, and a calculation error stops the process.

• *Non-Functional Requirements Mapped:* Payroll must be accurate, data must remain confidential, and processing must be efficient.


UC_55: Process Payroll
HR / Finance Staff
System
Select payroll period
Retrieve staff data
Missing data
Data exists
Trigger error message
Data complete
Calculate salaries (overtime/deductions)
Error detected
Calculation successful
Review payroll details
Stop process & trigger error
Record payroll information
Confirm completion

Airport Management System

System Administration Models Documentation

Prepared by:
Eneida Balliu

Course:
Software Modeling and Design

Date:
27 March 2026

Document Overview

This document presents a structured set of behavioral models for the System Administration module of the Airport Management System (AMS). It focuses on modeling the core administrative operations performed by the system administrator through Activity and State Machine diagrams.

The document includes the following system administration models:

✔ Manage User Accounts Activity Diagram (UC_40)
✔ Update Airport Information Activity Diagram (UC_41)
✔ Monitor System Usage Activity Diagram (UC_42)
✔ Manage System Permissions Activity Diagram (UC_43)
✔ Perform System Maintenance Activity Diagram (UC_44)
✔ Generate System Activity Reports Activity Diagram (UC_45)

Model Structure

Each behavioral model is developed based on its corresponding use case and includes:

1. Mapped Functional Requirements
2. Main and Alternative Flows Representation
3. Relevant Non-functional Constraints
4. Swimlane-Based Actor Interactions (System Administrator & System)

Purpose of the Document

The purpose of this document is to provide a clear, structured, and visual representation of the system's administrative workflows. These diagrams translate the functional requirements defined in the Software Requirements Specification (SRS) into logical and easy-to-understand process flows.

Each diagram has been designed following best practices in UML modeling, ensuring:

- simplicity and readability
- minimal and necessary decision points
- correct use of activity flow elements

3. System Administration Diagrams

Activity Diagram 1: Manage User Accounts (UC_40)

• Core Purpose:

This use case describes how the system administrator creates, updates, and deletes user accounts within the system.

• Functional Requirements Mapped:

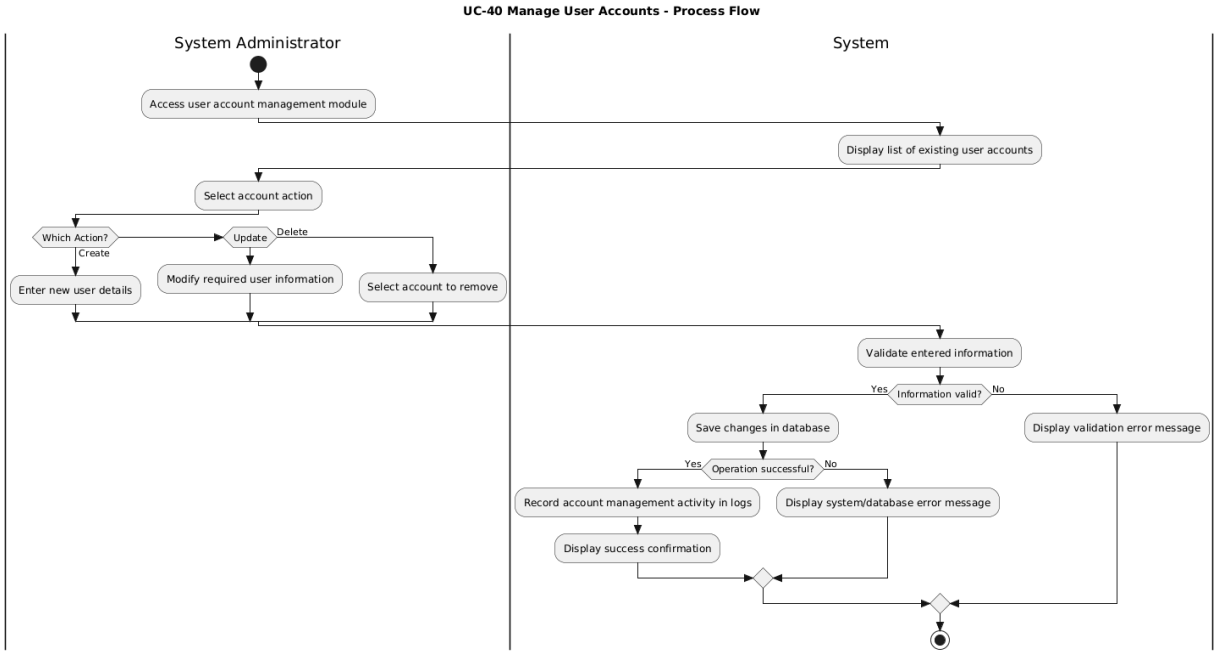
• System administrator accesses the user account management module.
• System displays a list of existing user accounts.
• Administrator selects an action (create, update, or delete).
• Administrator enters new user details, modifies existing information, or selects an account to delete.
• System validates the entered information.
• System saves changes in the database.
• System records account management activities in logs and confirms successful operation.

• Alternative Flow Mapped:

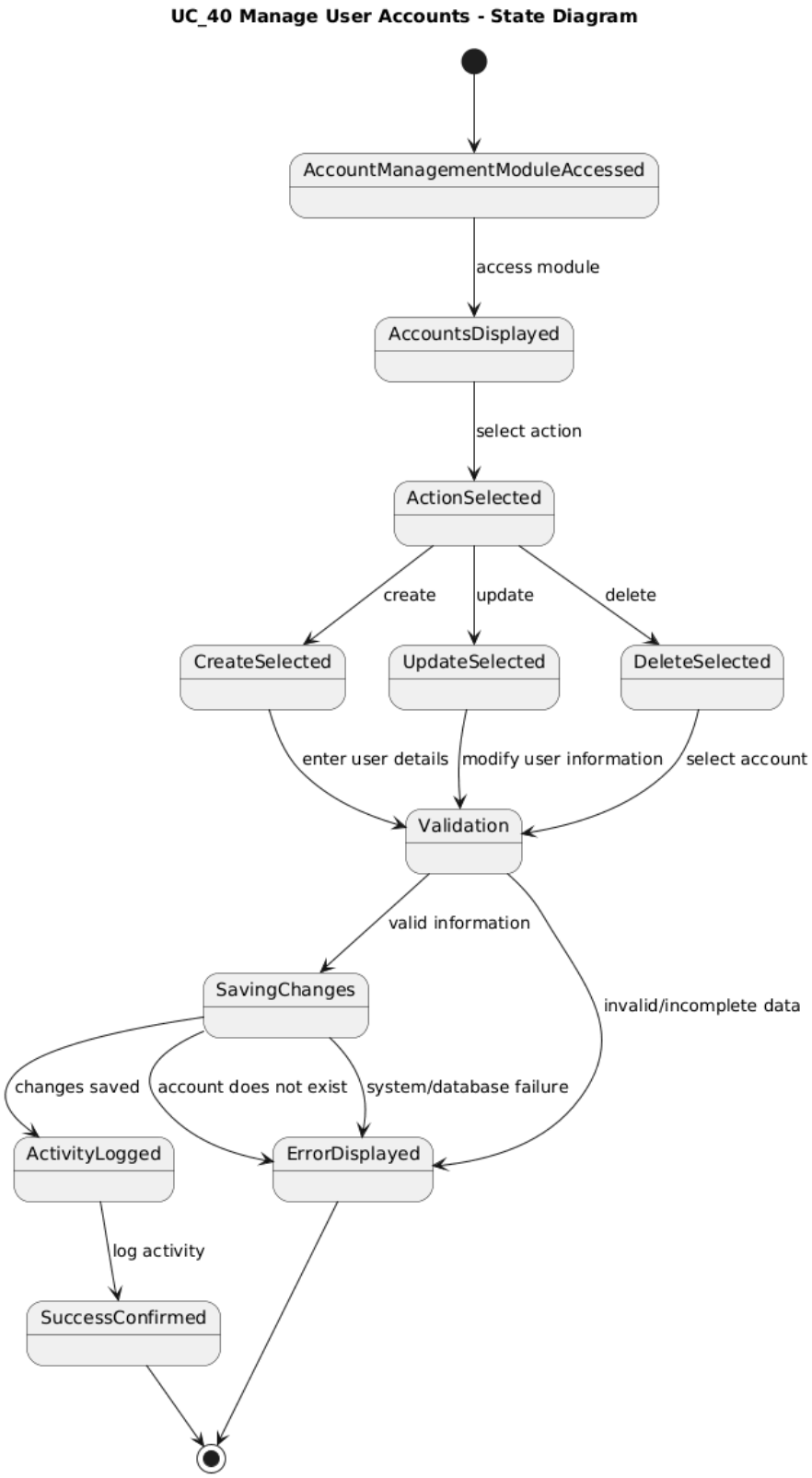
If invalid or incomplete data is entered, the system displays a validation error message. If the operation fails due to a system or database error, an error message is displayed. If the selected account does not exist, the system notifies the administrator.

• Non-Functional Requirements Mapped:

User data must be securely managed, role-based access control (RBAC) must be enforced, and all account management activities must be logged for auditing purposes.



State Diagram 1: Manage User Accounts (UC_40)



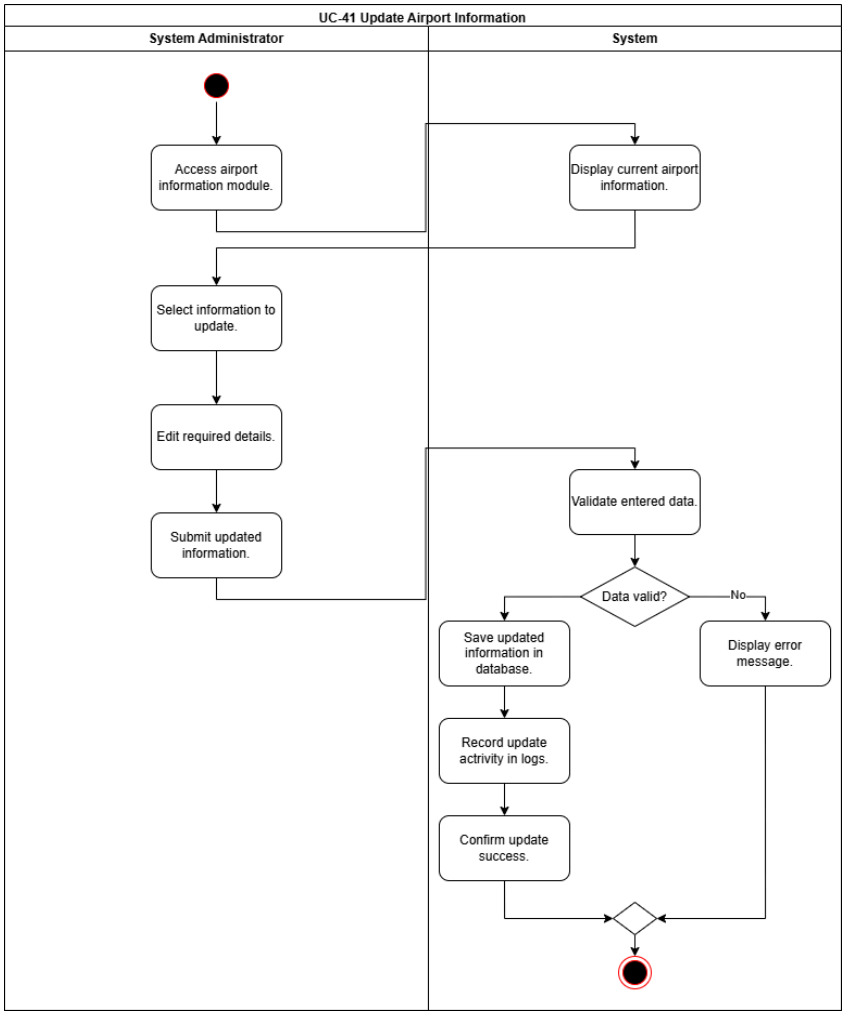
Activity Diagram 2: Update Airport Information (UC_41)

• *Core Purpose:*
This use case describes how the system administrator updates airport-related information within the system.

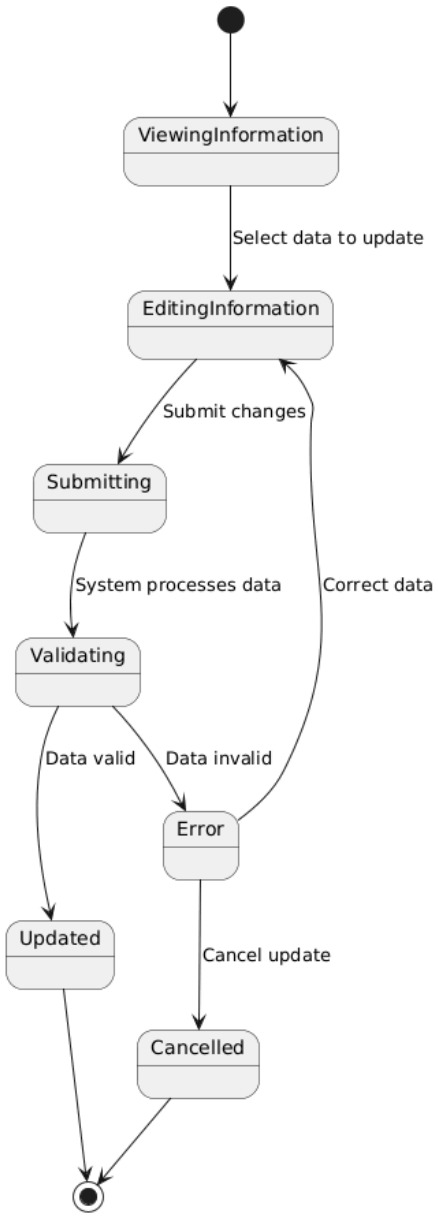
• *Functional Requirements Mapped:*
• System administrator accesses the airport information module.
• System displays current airport information.
• Administrator selects and edits the required details.
• Administrator submits updated information.
• System validates the entered data.
• System saves updated information in the database.
• System records update activity in logs and confirms success.

• Alternative Flow Mapped:
If invalid or incomplete data is entered, the system displays an error message. If the operation fails due to a system or database error, an error is displayed.

• *Non-Functional Requirements Mapped:*
System must ensure secure access, enforce role-based access control (RBAC), maintain data accuracy, and log all update activities.



State Diagram 2: Update Airport Information (UC_41)



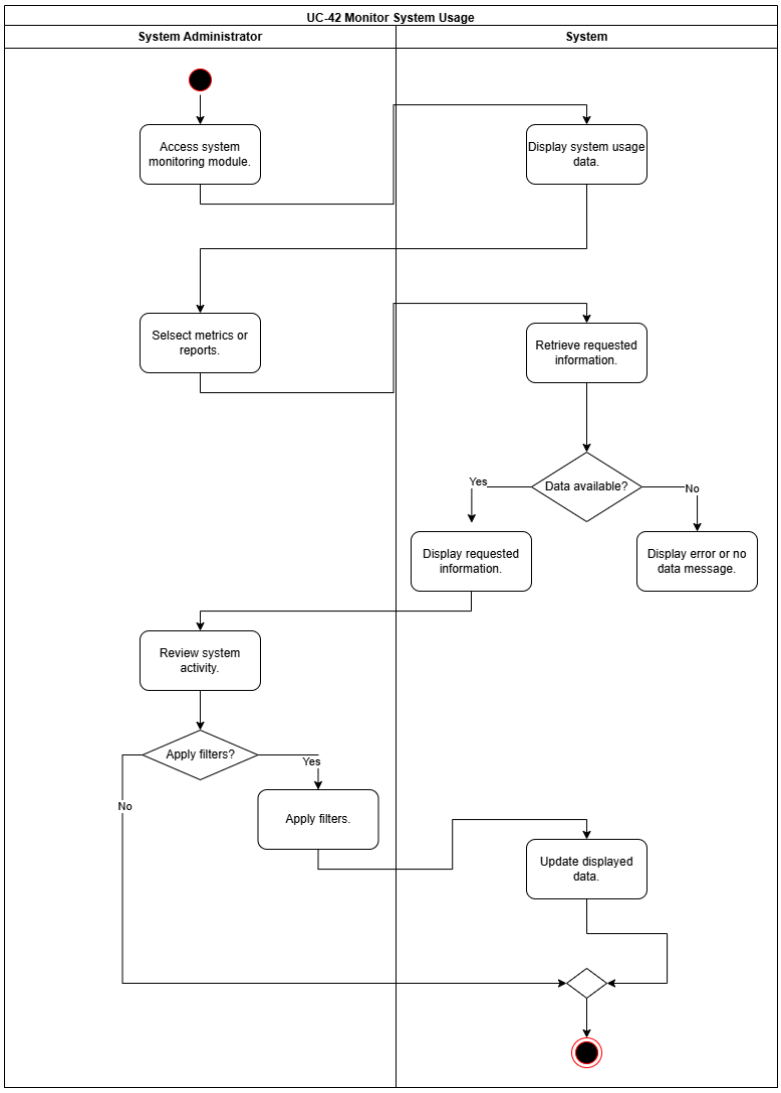
Activity Diagram 3: Monitor System Usage (UC_42)

• *Core Purpose:*
This use case describes how the system administrator monitors system activity, usage statistics, and performance.

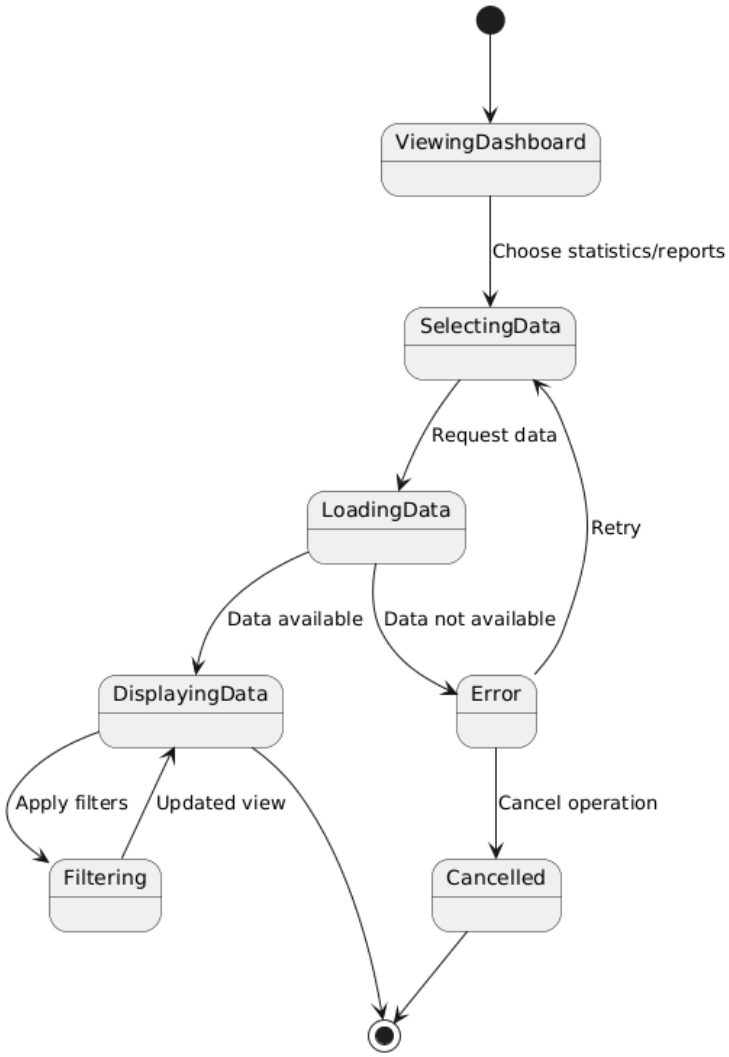
• *Functional Requirements Mapped:*
• System administrator accesses the system monitoring module.
• System displays system usage data.
• Administrator selects specific metrics or reports.
• System retrieves and displays the requested information.
• Administrator reviews system activity and may apply filters.
• System updates displayed data based on selected filters.

• *Alternative Flow Mapped:*
If data is unavailable or cannot be retrieved, the system displays an error or notification message.

• *Non-Functional Requirements Mapped:*
System must provide real-time data, ensure secure access, maintain accuracy, and display information within acceptable response time.



State Diagram 4: Manage System Permissions (UC_42)



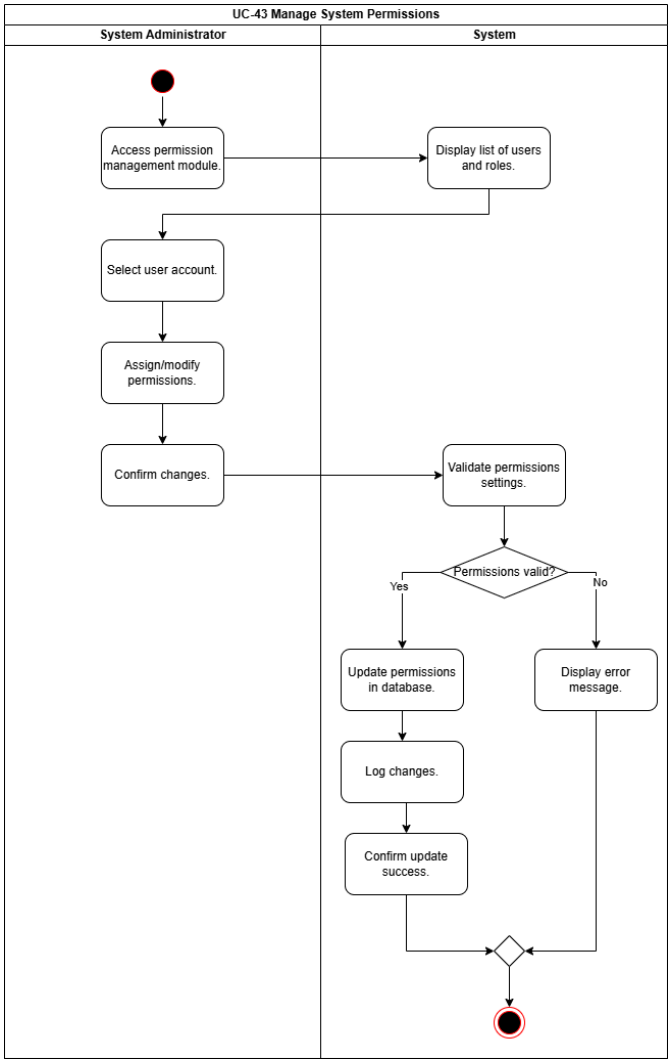
Activity Diagram 4: Manage System Permissions (UC_43)

• *Core Purpose:*
This use case describes how the system administrator manages user roles and access permissions.

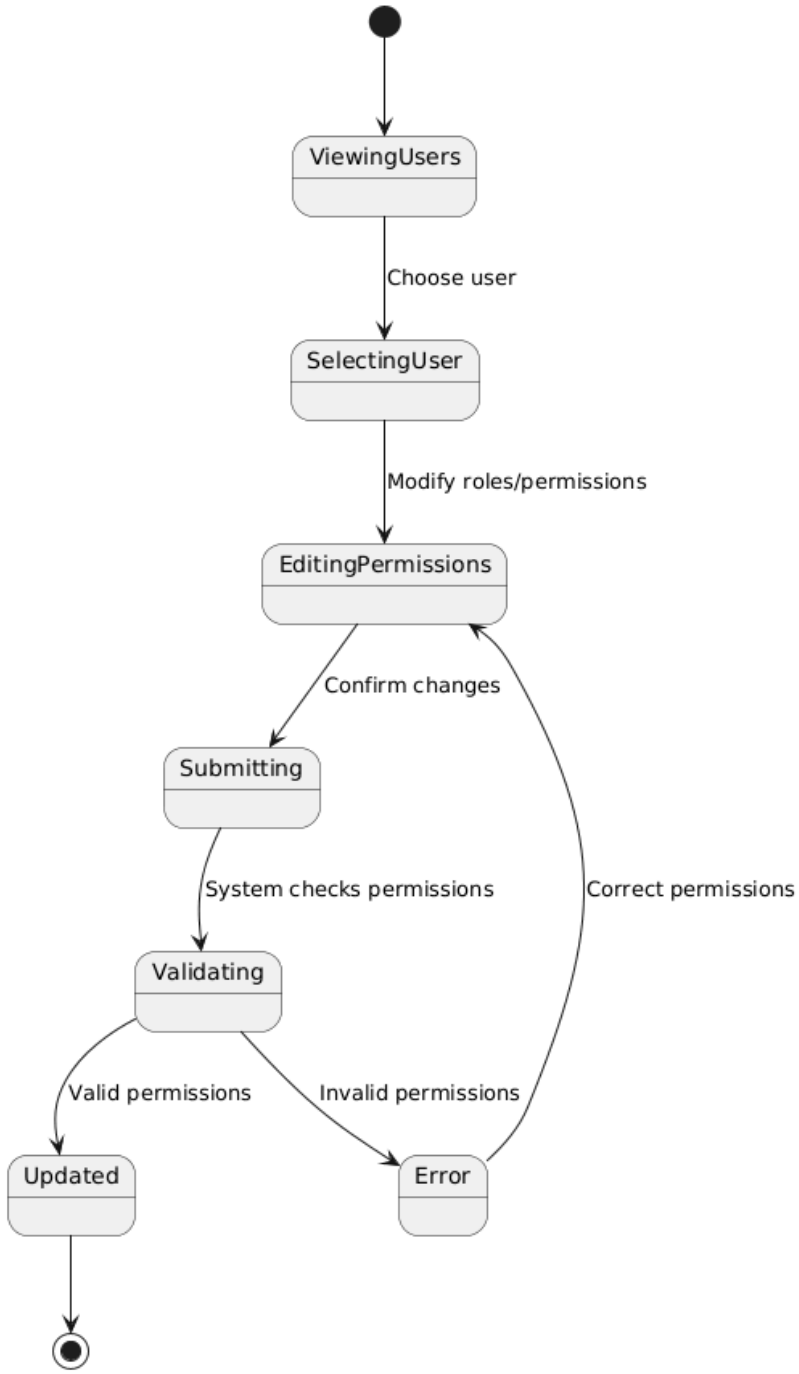
• *Functional Requirements Mapped:*
• System administrator accesses the permission management module.
• System displays users and their current permissions.
• Administrator selects a user and modifies roles or permissions.
• Administrator confirms the changes.
• System validates and updates permissions in the database.
• System confirms successful update.

• *Alternative Flow Mapped:*
If invalid permissions are assigned or the user does not exist, the system displays an error message.

• *Non-Functional Requirements Mapped:*
System must enforce RBAC, ensure secure access, maintain data integrity, and log all permission changes.



State Diagram 4: Manage System Permissions (UC_43)



Activity Diagram 5: Perform System Maintenance (UC_44)

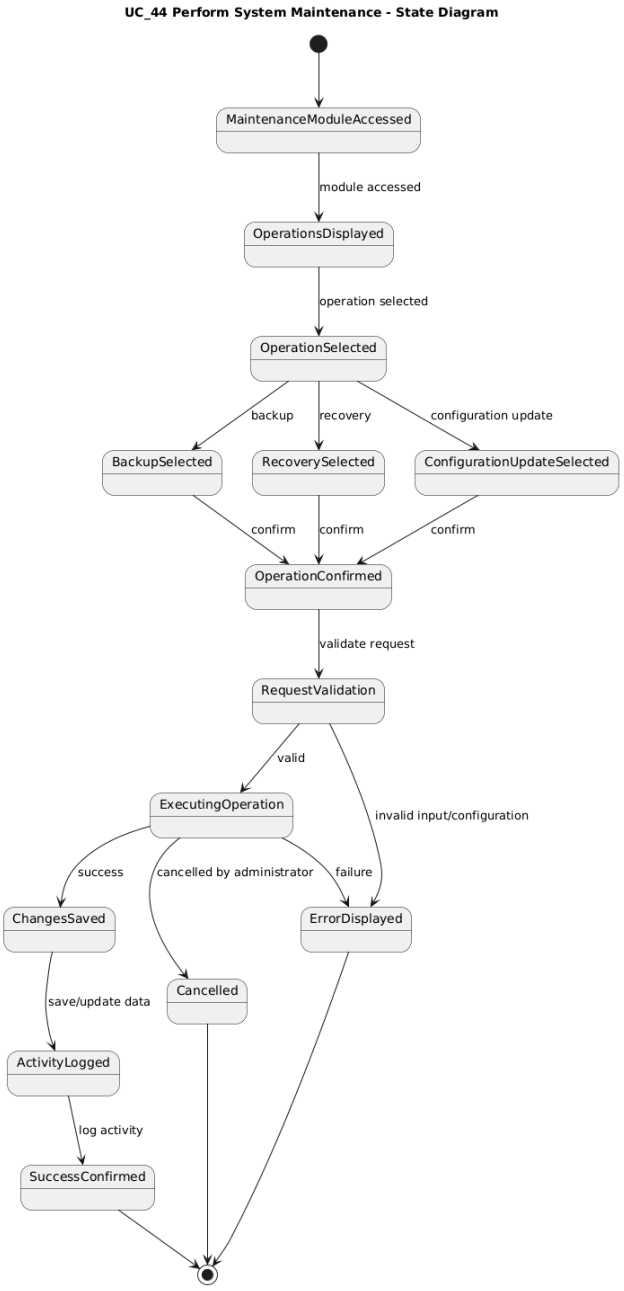
• *Core Purpose:*
This use case describes how the system administrator performs maintenance operations such as backup, recovery, and configuration updates.

• *Functional Requirements Mapped:*
• System administrator accesses the maintenance module.
• System displays available maintenance operations.
• Administrator selects an operation (backup, recovery, configuration update).
• Administrator confirms the operation.
• System validates and executes the selected operation.
• System saves changes and updates the database if necessary.
• System records maintenance activity and confirms completion.

• *Alternative Flow Mapped:*
If invalid input is provided or the operation fails, the system displays an error message.

• *Non-Functional Requirements Mapped:*
System must ensure data integrity, secure access, minimize downtime, and log all maintenance activities

State Diagram 5: Perform System Maintenance (UC_44)



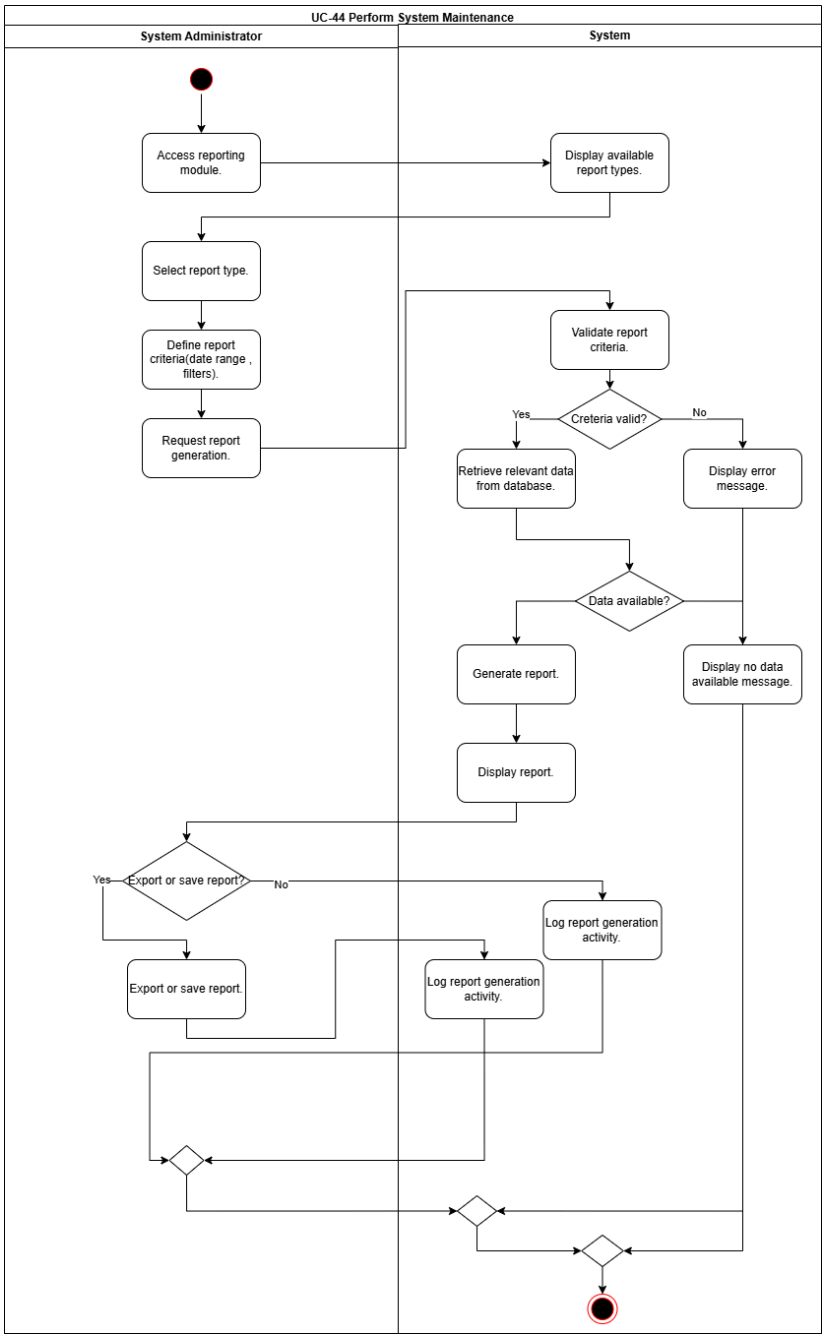
Activity Diagram 6: Generate System Activity Reports (UC_45)

• *Core Purpose:*
This use case describes how the system administrator generates and views system activity reports.

• *Functional Requirements Mapped:*
• System administrator accesses the reporting module.
• System displays available report types.
• Administrator selects report type and defines criteria.
• Administrator requests report generation.
• System retrieves data and generates the report.
• System displays the report.
• Administrator may export or save the report.

• *Alternative Flow Mapped:*
If no data is available or invalid criteria are entered, the system displays an error message.

• *Non-Functional Requirements Mapped:*
System must ensure report accuracy, secure access, efficient processing, and support exporting in standard formats.


UC-44 Perform System Maintenance
System Administrator
System
Access reporting module.
Display available report types.
Select report type.
Define report criteria(date range , filters).
Request report generation.
Validate report criteria.
Creteria valid?
Yes
No
Retrieve relevant data from database.
Display error message.
Data available?
Generate report.
Display no data available message.
Display report.
Export or save report?
Yes
No
Log report generation activity.
Export or save report.
Log report generation activity.

State Diagram 6: Generate System Activity Reports (UC_45)